

浙江工业大学

数据库系统课程设计报告

2025 短学期



小组 第 10 小组

题目 高校教务管理系统设计与实现

班级 软件工程 2304

学号 302023572097

姓名 田笑喧

指导教师 于明远

所在学院 计算机学院

提交日期 2025 年 7 月 8 日

目录

高校教务管理系统设计与实现	1
1 需求分析	2
1.1 数据需求描述	2
1.1.1 数据需求描述	2
1.1.2 数字字典	3
1.1.3 初级数据流图	5
1.2 系统功能需求	10
1.3 其他性能需求	11
2、概念结构设计	11
2.1 局部 E-R 图	12
2.2 全局 E-R 图	16
2.3 优化 E-R 图	17
3、逻辑结构设计	18
3.1 关系模式设计	18
3.2 数据类型定义	19
3.3 关系模式的优化	20
4、物理结构设计	21
4.1 聚簇设计	21
4.2 索引设计	21
4.3 分区设计	22
5、数据库实施	23
5.1 基本表建立	23
5.2 视图的建立	26
5.3 索引的建立	29
5.4 触发器建立	30
5.5 建存储过程	33
6、高校教务管理系统开发与运行	36

6.1 开发平台和开发环境介绍	36
6.2 数据准备.....	37
6.3 前台界面与后台数据库连接说明	38
6.4 应用系统的开发	41
6.4.1 系统各功能设计	41
6.4.2 应用系统结构	41
6.5 系统界面展示	42
6.5.1 系统管理员界面&教务管理界面	42
6.5.2 教师界面	45
6.5.3 学生界面	46
7、课程设计总结	47
7.1 遇到的问题和解决的办法	47
7.2 系统设计的不足	48
7.3 进一步改进思路和体会	48
7.4 致谢	49

高校教务管理系统设计与实现

罗可嘉 (小组组长)¹, 田笑喧²

¹(浙江工业大学计算机学院软件工程专业)

²(浙江工业大学计算机学院软件工程专业)

摘 要 本报告详细介绍了高校教务管理系统的设计与实现过程。该系统旨在实现学生信息、课程安排、成绩记录等核心功能的数字化管理，支持学分自动计算、成绩录入与查询，以及多维度统计分析。系统采用前后端分离架构，后端基于 Python 的 FastAPI 框架开发，前端使用 Vue.js 实现，数据库选用 GaussDB，确保了系统的可扩展性和高效性。通过角色权限控制，系统为管理员、教师和学生提供了差异化的功能模块，包括学生信息管理、成绩录入与查询、课程安排等。此外，报告还涵盖了数据库设计、物理结构优化、触发器与存储过程的实现，以及系统开发中遇到的问题与改进思路。

关键词 高校教务管理系统；数据库设计；FastAPI；GaussDB；权限控制；触发器；存储过程

本小组成员分工介绍

罗可嘉: 需求分析和逻辑结构设计，针对学生的基本表设计

田笑喧: 概念结构和物理结构设计，针对教师的基本表设计

1 需求分析

学生信息管理系统旨在实现学生档案、课程、成绩的数字化管理，以支持学分自动计算、成绩录入查询及多维度统计分析。系统应用于校园教学管理、学生自助查询及教师查询统计学生信息等数据报表，服务于管理员、教师、学生及教学管理人员等多类角色。从可行性来看，技术层面采用 Java Web 开发框架结合 GaussDB 数据库，使用数据库外键、触发器、存储过程等机制保障数据完整性与业务自动化；操作层面按角色分层设计简洁界面，通过成绩录入与学分计算的自动化关联减少人工误差；经济层面基于开源技术栈降低开发维护成本，提升办公效率。

1.1 数据需求描述

1.1.1 数据需求描述

本系统旨在实现学生信息的全面管理，包括学生基本信息、课程安排、教学关系、成绩记录以及用户登录控制等核心模块。为此，需设计和实现一套结构化的数据库来满足各类操作与功能需求。系统的数据需求描述如下：

- 地区信息

记录学生生源地的编号与名称，用于学生地域统计与查询。

- 系部信息

存储院系编号与名称，每个班级和学生均从属于某个系部，是组织教学和统计分析的基础单元。

- 班级信息

每个班级具有唯一编号，隶属于某一系部，班级信息用于管理学生归属及安排教学任务。

- 教师信息

包括教师工号、姓名、性别、年龄、职称与联系方式，用于支撑教学安排与教师权限管理。

- 学生信息

学生表为核心数据表，记录学号、姓名、性别、年龄、班级、系部、地区、入学时间、累计学分等基本信息，扩展字段包括联系方式与地址，支持成绩统计与身份验证等功能。

- 课程信息

存储课程编号、课程名称、学分、课时数及考核方式等，课程学分被限定在 1 至 10 之间，确保教学规范性。

- 教学安排

用于关联课程、教师、班级与学期，建立多对多教学关系，每条记录唯一标识一门课在某班某学期由某教师教授的安排。具有唯一性约束以防重复排课。

- 成绩记录

用于记录学生在特定教学任务下的成绩，成绩为 0 至 100 的浮点数，联合主键由学生编号与教学 ID 组成。

- 用户登录信息

提供系统登录功能，包括用户名、密码、角色（教师、学生、管理员等）、姓名与注册时间，用于身份识别与权限控制。

此外，系统对数据一致性、完整性进行了严格约束：

- 所有主表均设置主键，保证记录唯一性；
- 使用外键建立数据间关联，防止孤立记录；
- 对性别、成绩、学分等字段设置检查约束（CHECK），保证数据值域合法；
- 部分字段设置默认值，如学生学分默认 0，用户创建时间默认为当前时间；
- 通过触发器与约束机制，确保用户操作合法性与系统数据稳定性。

1.1.2 数字字典

1) 数据项

表 1 数据项

数据项名	描述
地区名	字符串类型，学生籍贯地区名
地区编号	字符串类型，作为地区唯一标识
班级名	字符串类型，学生所在班级
学生手机号	字符串类型，学生的联系方式
地址	字符串类型，学生家庭住址
学号	字符串类型，作为学生唯一标识
学生姓名	字符串类型，不可为空
班级编号	字符串类型，用于系统登录关联
专业编号	字符串类型，关联专业信息
专业名	标识学生所属专业名称
课程编号	课程唯一标识
课程名	课程完整名称
老师编号	教师唯一标识
老师姓名	教师真实姓名
老师所在系	教师所属系别
教师性别	教师的性别
教师手机号	教师的手机号
教师年龄	教师的年龄
职称	教师专业技术职称
成绩	学生课程成绩，范围 0-100
学期	课程开设的学期信息
学分	课程对应学分值

2) 数据结构

表 2 数据结构

数据结构名	组成数据项	描述
地区信息	包含地区 id 和地区名	用于记录学生籍贯地区
专业信息	包含专业 ID 和专业名	用于记录学校有哪些专业
班级信息	包含班级 ID、班级名和所属专业	用于记录班级信息
用户基本信息	包含 用户名、密码、角色类型	用于系统用户登录与权限管理
教师信息	包含教师名、教师 ID、教师手机号等信息	用于存储教师信息
学生信息	包含学号、姓名、性别、年龄、籍贯、班级编号、已修学分、地区编码、手机号码等	存储学生基础档案数据
课程信息	包含课程编号、名称、学分、课程类型、课时、学期、教师编号、班级编号等	记录课程详细信息
成绩信息	包含学号、课程编号、学期、成绩、教师编号等	存储学生课程成绩数据

3) 数据流

输入：学生档案数据、课程信息、成绩录入数据、查询请求等。

处理过程：数据验证、存储、计算（如学分自动统计，平均分计算）、查询逻辑处理等。

输出：学生信息报表、成绩查询结果、学分统计报表等。

4) 数据存储

表 3 数据存储

表名	主要字段
地区表 (Tianxx_Regions)	地区编码、地区名称
系别表 (Tianxx_Depts)	系别编号、系别名称
班级表 (Tianxx_Classes)	班级编号、班级名称、专业编号
教师表 (Tianxx_Teachers)	教师编号、姓名、性别、年龄、职称、电话
学生表 (Tianxx_Students)	学号、姓名、性别、年龄、籍贯、班级编号、 入学时间、已修学分、地区编码、手机号码
课程表 (Tianxx_Courses)	课程编号、名称、学分、课时、考察类型
教学表 (Tianxx_Teaching)	编号、课程编号、学生编号、教师编号、开 课学期
成绩表 (Tianxx_Reports)	学号、课程编号、成绩
用户表 (Tianxx_users)	用户名、密码、身份、显示名

1.1.3 初级数据流图

外部实体：

教师：提供登录凭证，查询和提交成绩数据

管理员：管理系统基础数据

学生：查询个人成绩信息

系统：高校学生成绩管理系统（中心处理单元）

数据流：

教师 → 系统：登录信息、查询请求、成绩录入/修改

管理员 → 系统：数据维护请求（学生/教师/课程信息的增删改查）

学生 → 系统：登录信息、成绩查询请求

具体如图 1-1 所示。

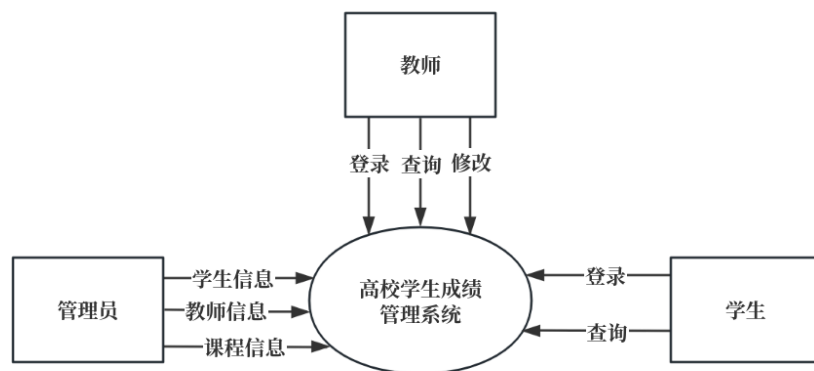


图 1 初级数据流图

1.1.4 详细级数据流图

● 学生模块

外部实体：

学生：查询个人成绩信息、登录与密码管理、信息查询

数据存储（开口矩形）

学生信息：存储学号、姓名、班级等。

用户信息：存储用户名称、密码等。

班级信息：记录班级名称、所属专业、学生名单等。

教师信息：存储教师姓名、授课课程、联系方式等。

课程信息：包括课程号、名称、学分、授课教师等。

授课表信息：包括授课老师、课程名等。

数据流：

(1) 登录与密码管理

学生 → 学生用户登录：输入 学号 和 密码。

学生用户登录 → 学生信息：验证密码。

学生用户登录 → 学生密码修改：提交新密码。

学生密码修改 → 学生信息：更新密码。

(2) 信息查询

学生 → 个人信息查询：请求个人数据（需 学号）。

个人信息查询 → 学生信息：返回姓名、班级等。

学生 → 查到班级课表：查询课表（需 班级 或 学号）。

查到班级课表 → 班级信息：获取课程安排。

学生 → 查到课程具体信息：输入 课程号。

查到课程具体信息 → 课程信息：返回课程详情。

(3) 成绩查询

学生 → 查到课程成绩：查询单科成绩（需 学号 和 课程号）。

查到课程成绩 → 校报考试信息：获取成绩数据。

学生 → 查询个人成绩统计：请求整体成绩报告。

查询个人成绩统计 → 校报考试信息：统计 GPA、学分等。

具体如图 1-2 所示。

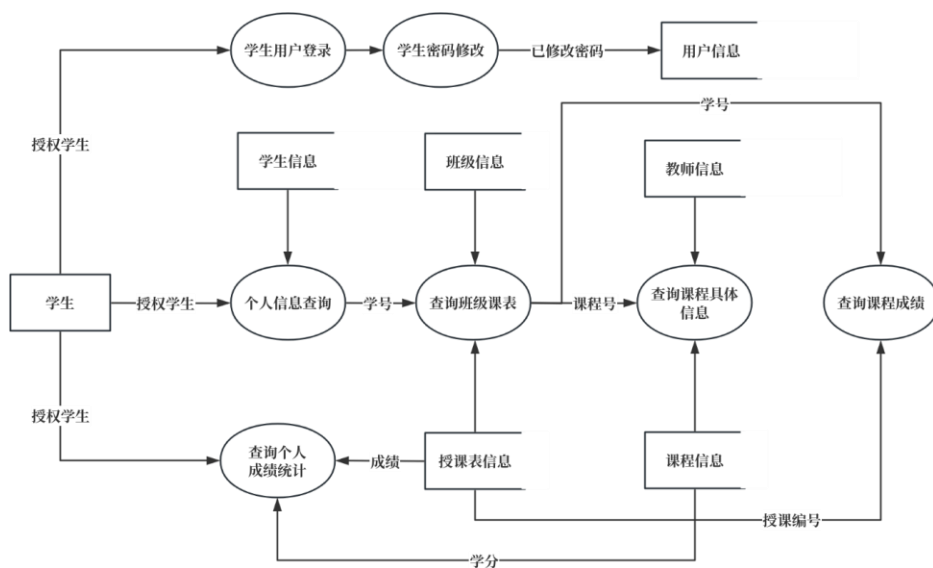


图 1 学生模块数据流图

● 教师模块

外部实体：

教师：管理学生成绩信息、登录与密码管理、信息查询

数据存储（开口矩形）

学生信息：存储学号、姓名、班级等。

用户信息：存储用户名称、密码等。

成绩信息：记录班级名称、所属专业、学生名单等。

教师信息：存储教师姓名、授课课程、联系方式等。

授课表信息：包括授课老师、课程名等。

数据流：

（1）登录流程

教师 → [教师用户登录] → 教师信息（验证）

（2）密码管理

教师 → [教师密码修改] → 教师信息（更新）

（3）信息查询

教师 → [个人信息查询] → 教师信息

教师 → [查询个人任课情况] → 授课表信息

教师 → [查询任课成绩统计] → 成绩信息

（4）成绩管理：

教师 → [更新学生成绩] → 成绩信息（需关联学生信息）

具体如图 1-3 所示。

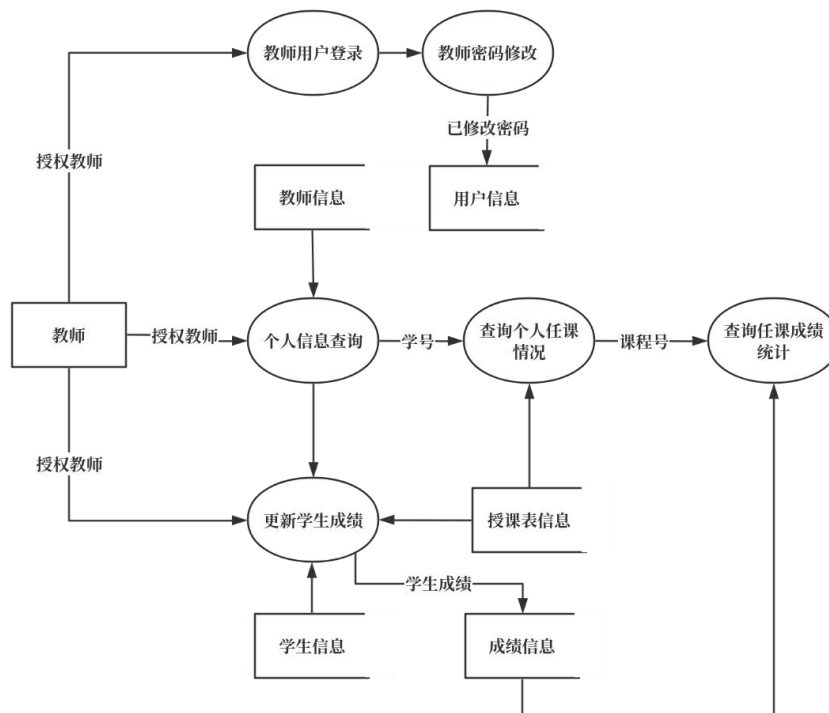


图 2 教师模块数据流

● 管理员模块

外部实体：

管理员：教师管理、登录与密码管理、学生管理、课程管理、生源地统计

数据存储（开口矩形）

学生信息：存储学号、姓名、班级等。

用户信息：存储用户名称、密码等。

成绩信息：记录班级名称、所属专业、学生名单等。

教师信息：存储教师姓名、授课课程、联系方式等。

授课表信息：包括授课老师、课程名等。

数据流：

（1）身份验证：

管理员 → [管理员登录] → 系统权限验证

（2）教师管理：

管理员 → [教师信息查询] → 输入教师编号 → 返回教师信息

管理员 → [教师信息修改] → 更新教师信息表

管理员 → [教师密码重置] → 重置教师密码

（3）学生管理：

管理员 → [学生信息查询] → 返回学生信息

管理员 → [学生信息修改] → 更新学生信息表

管理员 → [学生密码重置] → 重置学生密码

（4）统计功能：

管理员 → [统计生源地学生人数] → 生成统计报表

具体如图 1-4 所示。

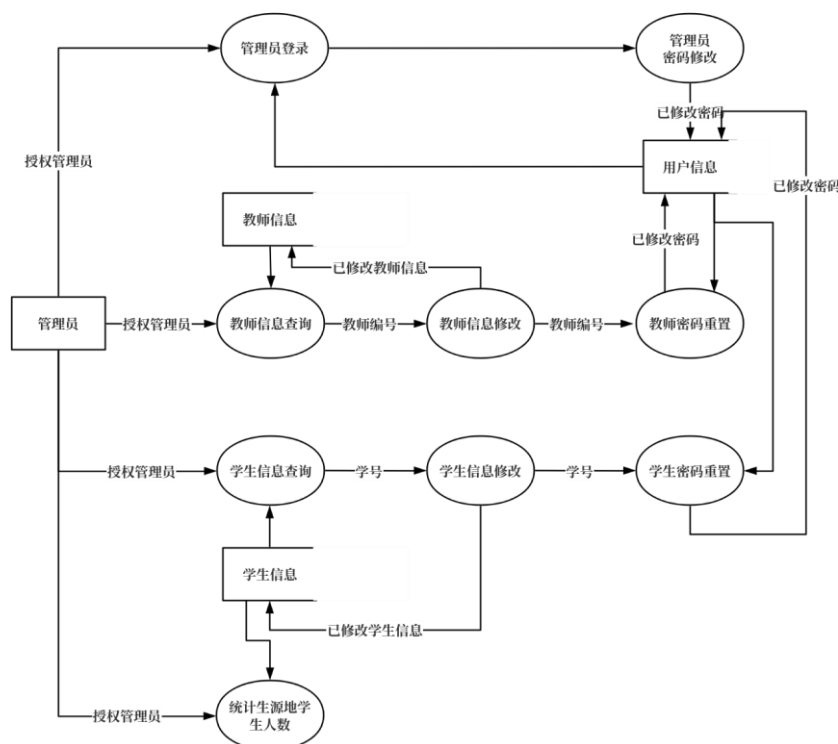


图3 管理员模块数据流

1.2 系统功能需求

本系统面向学生、教师和管理员三类用户，提供基于角色权限的学生信息管理服务。各类用户在系统中承担不同的职能，系统需支持多角色登录与操作控制，确保功能完整、权限清晰、数据一致。具体功能需求如下：

- 学生用户功能
 - 登录系统，查看个人基本信息（姓名、性别、班级、联系方式等）；
 - 修改部分信息，如联系电话、家庭地址等；
 - 按学期查询所修课程及成绩；
 - 查看已修学分总计；
 - 在线选课功能，供选修课程模块扩展使用。
- 教师用户功能
 - 登录系统，查看个人信息与授课安排；
 - 查询所授课程学生名单与成绩；
 - 录入与修改所教课程学生的成绩；
 - 查看学生的成绩分布与排名（班级内）；
 - 对教学任务（如所授课程）进行管理辅助功能（可拓展为审核或评论模块）。
- 管理员功能
 - 管理学生信息，包括添加、删除、修改和查询；
 - 管理教师信息，包括基本信息录入、修改和删除；
 - 管理课程信息，包括课程开设、课时、学分、考核方式等内容；
 - 管理班级、系部、地区等基础信息；
 - 设置和维护教学安排，如教师授课班级、授课课程和学期安排；
 - 查看全校学生成

绩数据，生成统计报表；

- 管理系统用户权限与账户信息。
- 公共功能需求
 - 提供基于角色的权限控制与身份验证；
 - 提供友好的用户界面，实现高效的人机交互；
 - 系统应具备数据导入导出功能，支持 Excel/CSV 格式的批量操作；
 - 提供日志记录机制，追踪关键操作行为； - 保证系统数据一致性、完整性和安全性。

1.3 其他性能需求

本系统除了需满足基本的功能性需求，还必须具备良好的非功能特性，以保障系统在实际运行中的可靠性、扩展性和安全性。非功能需求主要包括以下几个方面：

- 性能需求
 - 系统应支持至少 1000 名学生、100 名教师的并发访问；
 - 所有查询操作响应时间应控制在 2 秒以内；
 - 大批量数据导入、导出（如学生信息、成绩）应在 10 秒内完成。
- 可用性需求
 - 系统应提供简洁直观的图形化用户界面，便于非技术用户操作；
 - 系统应具备异常处理机制，确保在输入错误、网络故障等情况下仍能稳定运行。
- 安全性需求
 - 所有用户必须通过用户名和密码登录系统；
 - 不同角色用户（管理员、教师、学生）应有不同的访问权限； - 密码在数据库中应加密存储；
 - 对敏感操作（如删除数据、成绩修改）应记录日志以备追溯； - 系统应防止常见网络攻击，如 SQL 注入、XSS 攻击等。
- 可维护性需求
 - 系统代码应结构清晰、模块化，方便后期维护和升级；
 - 数据库设计应遵循规范化原则，便于扩展字段和表结构；
 - 提供完整的日志和错误报告功能，辅助问题排查与系统调试。
- 可靠性需求
 - 系统应具备高容错能力，关键操作过程中出现异常时应能自动回滚；
 - 数据库应定期备份，防止数据丢失；
 - 应用程序应具备异常恢复机制，如服务器重启后能自动恢复服务。

2、概念结构设计

2.1 局部 E-R 图

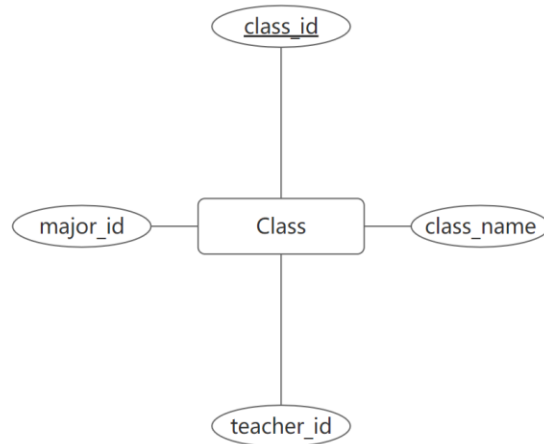


图 5 班级 E-R 图

● 班级表

班级编号 (class_id): 主键, 格式约束: 'C'+4 位数字

班级名称 (class_name): 非空, 唯一约束, 格式: "专业名称+入学年份+班号" (如"计算机 2301")

班主任 (teacher_id): 外键, 关联教师表 (允许 NULL)

专业编号 (major_id): 外键, 关联专业表 (删除约束: CASCADE)

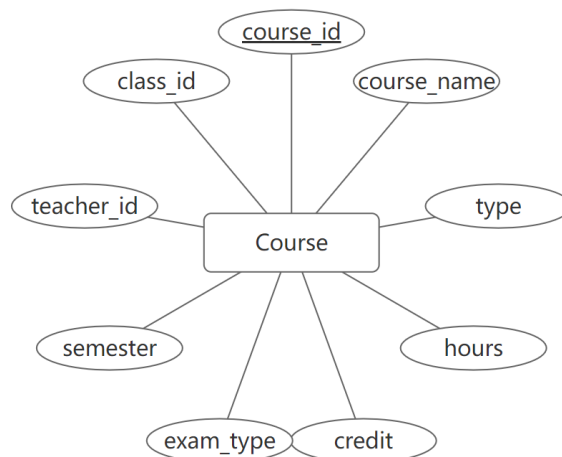


图 6 课程 E-R 图

● 课程表

课程编号 (course_id): 主键, 格式约束: 'K'+5 位数字

课程名称 (course_name): 非空, 长度≤50 字符

课程类型 (type): 必修/选修/通识

课程学时 (hours): 整数

课程学分 (credit): 数值

考试类型 (exam_type): 考试/考查

开课学期 (semester): YYYY-YYYY-[1|2] (如 2023-2024-1)

开课教师 (teacher_id): 外键, 关联教师表 (级联更新)

开课班级 (class_id): 外键, 关联班级表 (删除约束: CASCADE)

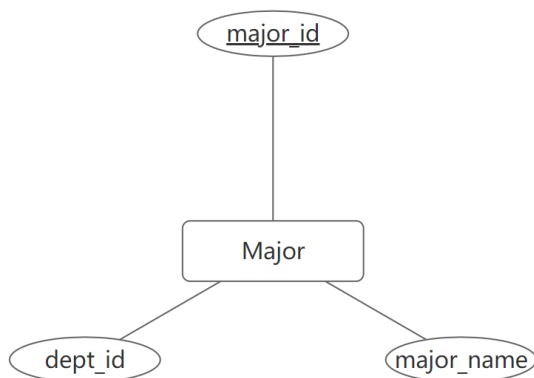


图 7 专业 E-R 图

● 专业表

专业编号 (major_id): 主键, 格式约束: 'M'+3 位数字

专业名称 (major_name): 非空

所属院系 (dept_id): 外键, 关联院系表 (级联更新)

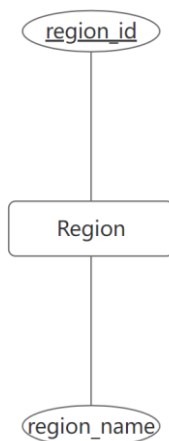


图 8 区域 E-R 图

● 地区表

地区编号 (region_id): 主键

地区名称 (region_name): 非空

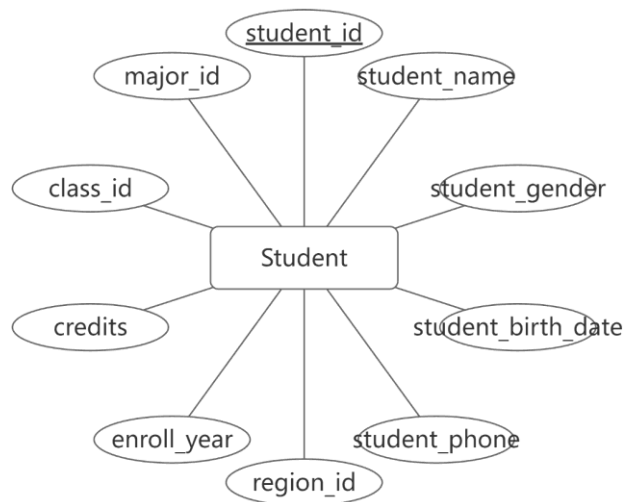


图 9 学生 E-R 图

● 学生表

学号 (student_id): 主键, 格式约束: 固定 7 位, 'S'+6 位数字

姓名 (student_name): 非空, 长度≤20 字符

性别 (student_gender): 只允许'男'或'女'

出生日期 (student_birth_date): 日期格式: 1990-01-01

联系方式 (student_phone): 11 位手机号

生源所在地 (region_id): 外键, 关联地区表 (级联更新)

入学年份 (enroll_year)

已修学分总数 (credits): 默认值 0

班级编号 (class_id): 外键, 关联班级表 (删除约束: RESTRICT)

专业编号 (major_id): 外键, 关联专业表 (级联更新)

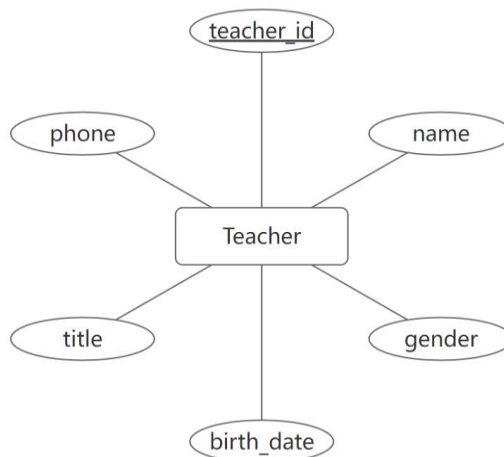


图 10 教师 E-R 图

● 教师表

教师编号 (teacher_id): 主键, 格式约束: 'T'+5 位数字

姓名 (name): 非空

性别 (gender): 男/女

出生日期 (birth_date): 如 1950-01-01

职称 (title): 助教/讲师/副教授/教授

联系方式 (phone): 同学生表

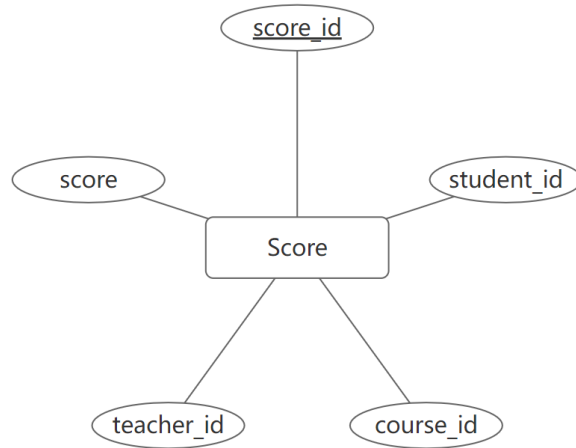


图 11 成绩 E-R 图

● 成绩表

成绩 ID (score_id): 自增主键

学号 (student_id): 外键, 关联学生表 (删除约束: CASCADE)

课程编号 (course_id): 外键, 关联课程表 (级联更新)

教师编号 (teacher_id): 外键, 关联教师表

综合成绩 (score): 数值, 当 $\text{score} \geq 60$ 时, 自动累加学分到学生表

约束: (student_id, course_id) 组合唯一

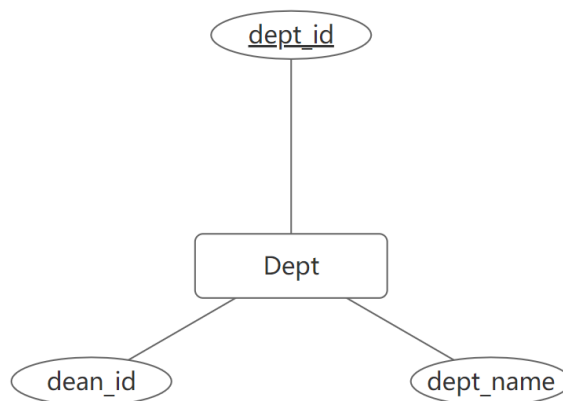


图 12 系表 E-R 图

● 院系表

院系编号主键, 格式约束: 'D'+2 位数字

院系名称 (dept_name): 非空

院长 (dean_id): 外键, 关联教师表 (允许 NULL)

2.2 全局 E-R 图

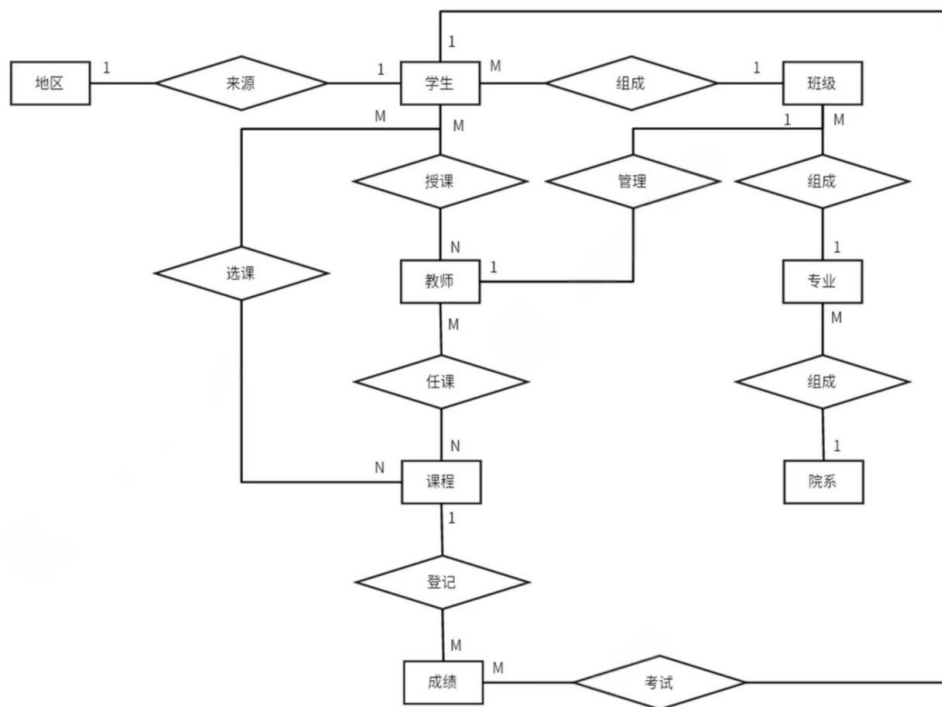


图 13 全局 E-R 图

1. 核心实体

学生 (Student)

属性：学号（主键）、姓名、性别、班级、联系方式等。

关联：属于一个班级（1 对多），来自一个地区（1 对多），参与多门课程的学习（多对多）。

•教师 (Teacher)

属性：教师编号（主键）、姓名、职称、联系方式等。

关联：教授多门课程（多对多），可能担任班主任（1 对 1）。

•课程 (Course)

属性：课程编号（主键）、名称、学分、考核方式等。

关联：由教师授课（多对多），被学生选修（多对多）。

•成绩 (Score)

属性：成绩 ID（主键）、分数、学期等。

关联：记录学生与课程的关联（多对多），依赖教学安排。

•地区 (Region)

属性：地区编号（主键）、名称。

关联：学生的生源地（1 对多）。

•考试 (Exam)（图中可能隐含）

属性：考试 ID、时间、地点。

关联：与课程绑定（1 对多）。

2. 实体间关系

- 学生—课程（多对多）

通过“成绩”实体实现，记录学生的课程分数。

- 教师—课程（多对多）

通过“授课”关系实现，表示教师可教授多门课程。

- 学生—地区（多对 1）

学生属于某一地区，地区可包含多名学生。

- 课程—考试（1 对多）

一门课程可能包含多次考试（如期中、期末）。

3. 关键约束

- 成绩的传递依赖

成绩由学生 ID 和课程 ID 共同决定，确保唯一性。

- 级联操作

删除学生时，自动删除其成绩记录（通过触发器实现）。

- 外键关联

如班级表关联院系表，确保数据完整性。

4. 图示补充说明

- 符号含义

1: 一端（如一个地区对应多个学生）。

M: 多端（如多个学生属于一个班级）。

N: 多对多（如教师与课程）。

2.3 优化 E-R 图

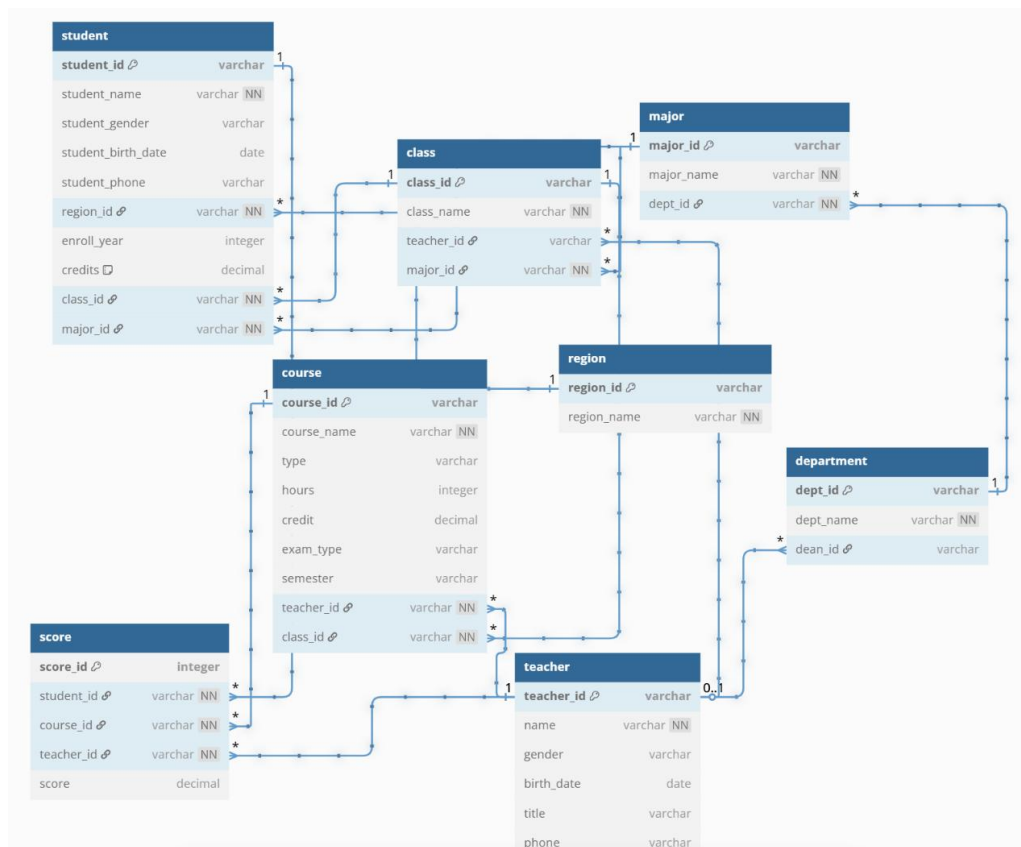


图 14 优化 E-R 图

优化点

1. **外键约束**: 所有外键均标注为非空 (NN)，确保数据完整性。
2. **主键设计**: 采用具有业务意义的字段（如 `student_id`、`course_id`）作为主键。
3. **关系简化**: 通过成绩表解决学生与课程的多对多关系，避免冗余。
4. **层级清晰**: 院系—专业—班级—学生的层级关系明确，便于管理。

3、逻辑结构设计

3.1 关系模式设计

将 E-R 图转换为关系模式，定义实体型、属性及其联系。

本系统采用关系型数据库进行数据存储，依据实际业务需求与实体之间的关系，设计了如下关系模式。各个关系表均具备完整的主键约束、外键约束与检查条件，确保数据一致性与完整性。主要关系模式如下所示：

• 地区表 (Tianxx_Regions)

Tianxx_Regions(txx_RegionID, txx_RegionName)

• 系部表 (Tianxx_Depts)

Tianxx_Depts(txx_DeptNo, txx_DeptName)

• 班级表 (Tianxx_Classes)

Tianxx_Classes(txx_ClassID, txx_DeptNo, txx_ClassName)

• 教师表 (Tianxx_Teachers)

Tianxx_Teachers(txx_Tno, txx_Tname, txx_Sex, txx_Age, txx_Title, txx_Phone)
--

• 学生表 (Tianxx_Students)

Tianxx_Students(txx_Sno, txx_Sname, txx_Sex, txx_Age, txx_RegionID, txx_DeptNo, txx_ClassID, txx_StartDate, txx_CreditHours, txx_Phone, txx_Address)
--

• 课程表 (Tianxx_Courses)

Tianxx_Courses(txx_Cno, txx_Cname, txx_CreditHour, txx_Hour, txx_AssessType)

• 教学安排表 (Tianxx_Teaching)

Tianxx_Teaching(txx_TeachingID, txx_Cno, txx_Tno, txx_ClassID, txx_Term)

• 成绩记录表 (Tianxx_Reports)

Tianxx_Reports(txx_Sno, txx_TeachingID, txx_Grade)

• 用户表 (Tianxx_users)

Tianxx_users(txx_id, txx_username, txx_password, txx_role, txx_name)

上述关系模式中，所有外键字段均已通过外键约束连接至对应主表，确保各实体之间关系的一致性和完整性。例如，txx_DeptNo 在学生表中引用了系部表，txx_TeachingID 在成绩表中引用了教学安排表。

此外，对某些字段设置了检查约束（如成绩范围、学分范围、性别限定）与默认值（如学生初始学分为 0、用户创建时间为当前时间），进一步保障系统的数据质量与运行安全。

3.2 数据类型定义

为满足学生信息管理系统中各类数据的表示需求，系统采用关系型数据库并基于字段的语义合理选用了不同的数据类型。在设计过程中，充分考虑了字段内容的长度、格式、约束要求以及查询性能，具体数据类型定义如下：

● 字符型 (CHAR, VARCHAR)

- **CHAR(n)**：用于长度固定的字符串，如学号 (CHAR(8))、教师编号 (CHAR(6))、地区编号 (CHAR(5)) 等。这些字段长度一致、格式规范，使用 CHAR 类型可提升存储和比较效率。
- **VARCHAR(n)**：用于长度不固定但有最大限制的字符串，如姓名 (VARCHAR(50))、课程名称 (VARCHAR(100))、考核方式 (VARCHAR(10))、地址 (VARCHAR(200)) 等。该类型节省存储空间，适用于可变长度文本。

● 整型 (INT, SERIAL)

- **INT**：用于存储整数数据，如年龄 (txx_Age)、课程课时 (txx_Hour)、课程学分 (txx_CreditHour)、累计学分 (txx_CreditHours) 等。
- **SERIAL**：用于系统自动生成的自增主键，如教学安排编号 (txx_TeachingID) 和用户编号 (txx_id)，简化插入操作，确保主键唯一性。

● 浮点/定点数型 (DECIMAL)

- **DECIMAL(5, 2)**：用于表示学生成绩 (txx_Grade)，支持最多 3 位整数和 2 位小数，确保成绩精度，适用于金融或成绩类数据，避免浮点误差。

● 布尔型

- 系统通过 CHECK 约束实现枚举类型约束，如性别字段限定为‘男’或‘女’，

课程学分限定在 1~10 之间，成绩值限定为 0~100 之间。

- 约束类型
 - **PRIMARY KEY**: 用于唯一标识每一条记录，如学号、教师编号、教学 ID 等；
 - **FOREIGN KEY**: 用于建立实体之间的引用关系，如学生所属班级、教师授课课程；
 - **UNIQUE**: 用于防止重复记录，如用户名字段；
 - **CHECK**: 用于限制取值范围，如成绩、性别、课程学分等；
 - **DEFAULT**: 为字段提供默认值，如学生学分初始为 0、用户注册时间默认当前时间。

通过合理设计数据类型与约束机制，不仅确保了数据的合法性与一致性，也提升了系统的运行效率与可维护性，为后续系统扩展和数据分析提供了坚实的数据结构基础。

3.3 关系模式的优化

在关系数据库设计中，为了提升数据一致性、查询效率与可维护性，需要对初始关系模式进行规范化与优化处理。本系统在分析业务逻辑与实体关系的基础上，结合数据库范式原则，完成了以下优化措施：

- 遵循数据库规范化原则
 - 所有关系模式均满足第三范式（3NF），即：每个非主属性完全依赖于主码，且不存在传递依赖；
 - 例如，学生表 `Tianxx_Students` 中，班级编号（`txx_ClassID`）作为外键引用班级表，避免冗余存储班级名称等属性；
 - 教学安排表 `Tianxx_Teaching` 拆分了教师、课程与班级之间的多对多关系，避免信息重复。
- 主键与外键设计优化
 - 所有实体表（如学生、教师、课程等）均设计了具有业务语义的主键字段；
 - 所有外键字段添加了参照完整性约束，确保引用字段必须存在于主表中，有效防止“悬挂记录”；
 - 对教学安排表、成绩表使用复合主键或自增 ID，明确记录的唯一性。
- 数据完整性与约束优化
 - 对性别、课程学分、成绩等字段设置 **CHECK** 约束，防止非法数据输入；
 - 设置默认值（如学分初始值为 0、注册时间为当前时间），提高数据插入效率；
 - 通过 **NOT NULL** 限定关键字段必须填写，提升数据质量。
- 关系冗余最小化
 - 学生信息中仅保存班级编号、地区编号等外键字段，不冗余存储地区名称、班级名称等信息；
 - 用户表与教师/学生编号保持一致，避免重复设计账号系统；
 - 成绩表与教学表分离，避免课程、教师、学生三者之间多对多关系带来的冗余问题。
- 可扩展性设计
 - 用户表通过角色字段（`role`）支持多类身份，便于未来扩展其他角色（如教务、辅

导员)；

- 教学安排表中添加学期字段 (txx_Term)，支持跨学期教学管理；
- 所有主键均设计为具有业务意义或自增型，便于系统扩展与接口调用。

通过以上优化，本系统的数据模型在结构清晰、约束完整的同时，也具备较高的扩展性与查询效率，为后续系统开发与维护打下了坚实的基础。

4、物理结构设计

4.1 聚簇设计

聚簇 (Cluster) 设计是数据库物理设计中的重要环节，旨在通过对相关数据的物理存储进行聚集，优化数据访问效率，尤其是在涉及大量关联查询和范围查询时表现突出。针对本系统的学生信息管理数据库，进行了以下聚簇设计考虑：

- 聚簇索引选择

- 在班级表 Tianxx_Classes 中，以主键 txx_ClassID 建立聚簇索引，保证同一班级的记录物理上连续存储，方便按班级查询学生及教学安排。
- 教学安排表 Tianxx_Teaching 的聚簇索引设计为自增主键 txx_TeachingID，配合外键字段实现高效的课程-教师-班级联合查询。

- 数据聚簇原则

- 学生表 Tianxx_Students 可基于班级编号 txx_ClassID 进行数据聚簇，以提高同班学生的存取局部性，减少磁盘 I/O。
- 成绩表 Tianxx_Reports 由于涉及学生和教学安排的联合查询，可通过复合索引实现部分聚簇效果，提升成绩查询和统计的响应速度。

- 聚簇设计带来的优势

- 降低了关联查询的磁盘寻址次数，提升查询效率，尤其是常见的“学生-班级-课程-成绩”多表联合查询。
- 改善缓存命中率，提高系统整体性能。
- 有利于数据的顺序扫描，方便统计分析和报表生成。

- 设计限制与注意事项

- 聚簇索引对插入和更新操作存在一定开销，设计时需权衡读写性能；
- 由于每张表只能有一个聚簇索引，需根据系统核心查询路径合理选择；
- 定期监控和维护聚簇结构，防止数据碎片影响性能。

4.2 索引设计

为了提升数据库的查询效率，减少数据检索的响应时间，本系统针对各主要表和关键查询字段设计了合理的索引策略。索引的设计遵循以下原则：

- 主键索引
 - 各表的主键字段均自动创建唯一索引，如学生表的 `txx_Sno`、教师表的 `txx_Tno`、课程表的 `txx_Cno` 等，保障数据唯一性及快速定位。
- 外键索引
 - 对于频繁用于连接查询的外键字段，如学生表中的 `txx_DeptNo`、`txx_ClassID`，教学安排表中的 `txx_Cno`、`txx_Tno`，均建立非唯一索引，加快关联查询性能。
- 联合索引
 - 在成绩表 `Tianxx_Reports` 中，因查询经常基于学生编号与教学安排编号联合过滤，设计联合索引 (`txx_Sno`, `txx_TeachingID`)，提高多条件查询效率。
 - 在教学安排表 `Tianxx_Teaching` 上，基于课程编号、教师编号、班级编号及学期字段联合建立索引，支持多维度教学信息快速检索。
- 文本字段索引
 - 对教师姓名、学生姓名等常用搜索字段建立前缀索引，提升模糊查询性能。
- 索引维护与优化
 - 定期监控索引的使用情况，避免不必要的索引冗余。
 - 对频繁更新的表合理设计索引，平衡读写性能。
 - 结合查询日志优化索引结构，针对热点查询调整索引策略。

合理的索引设计有效减少了全表扫描次数，加速了数据访问，提升了系统整体性能和用户体验。

4.3 分区设计

数据为了提高大规模数据的管理效率和查询性能，本系统在数据库设计中采用了合理的分区策略。分区能够将大表按照一定规则划分为多个逻辑独立但物理关联的数据块，从而提升数据的维护性和访问速度。具体设计如下：

- 分区依据
 - 以教学安排表 `Tianxx_Teaching` 中的学期字段 `txx_Term` 作为分区键，将教学数据按学期划分，便于按学期快速查询和归档历史数据。
 - 学生成绩表 `Tianxx_Reports` 可依据教学安排 ID `txx_TeachingID` 进行范围或列表分区，配合教学表分区优化联合查询。
- 分区类型选择
 - 采用范围分区 (`Range Partitioning`) 对学期字段进行划分，如按学年或学期段划分，支持按时间区间的高效查询。
 - 在学生信息量大时，针对学生表 `Tianxx_Students` 可考虑按学院编号 `txx_DeptNo` 做哈希分区，均衡数据分布。

- 分区优势
 - 减少单表数据量，降低查询时扫描数据的范围，提高响应速度。
 - 分区表可独立维护与备份，简化数据管理和归档。
 - 支持并行查询，提升系统整体吞吐量。

- 设计注意事项
 - 分区字段应选择访问频率高且分布均匀的字段，避免数据倾斜。
 - 分区设计需兼顾业务需求与系统性能，防止过度分区带来的管理复杂性。
 - 结合索引策略，保证分区表查询依然高效。

通过合理的分区设计，系统能够有效支持大数据量的管理需求，提高查询效率和系统的可扩展性。

5、数据库实施

5.1 基本表建立

根据系统的数据需求与关系模式设计，本节介绍数据库中各基本表的创建语句及关键字段说明，确保系统数据结构完整且满足业务逻辑需求。

◆ 地区表(Tianxx_Regions)

存储地区信息，主键为地区编号，确保地区名称唯一性。

```
CREATE TABLE Tianxx_Regions (  
    txx_RegionID CHAR(4) PRIMARY KEY,  
    txx_RegionName VARCHAR(100) NOT NULL  
);
```

◆ 系表(Tianxx_Depts)

存储系部信息，主键为系部编号，包含系部名称。

```
CREATE TABLE Tianxx_Depts (  
    txx_DeptNo CHAR(4) PRIMARY KEY,  
    txx_DeptName VARCHAR(50) NOT NULL  
);
```

◆ 班级表(Tianxx_Classes)

存储班级信息，关联系部表，主键为班级编号。

```
CREATE TABLE Tianxx_Classes (  
    txx_ClassID CHAR(6) PRIMARY KEY,  
    txx_DeptNo CHAR(4),  
    txx_ClassName VARCHAR(50) NOT NULL,  
    FOREIGN KEY (txx_DeptNo) REFERENCES Tianxx_Depts(txx_DeptNo)  
);
```

◆ 教师表(Tianxx_Teachers)

存储教师基本信息，主键为教师编号。

```
CREATE TABLE Tianxx_Teachers (  
    txx_Tno CHAR(6) PRIMARY KEY,  
    txx_Tname VARCHAR(50) NOT NULL,  
    txx_Sex CHAR(2),  
    txx_Age INT,  
    txx_Title VARCHAR(50),  
    txx_Phone VARCHAR(20)  
);
```

◆ 学生表(Tianxx_Students)

存储学生信息，关联地区、系部及班级，主键为学号。

```
CREATE TABLE Tianxx_Students (  
    txx_Sno CHAR(8) PRIMARY KEY,  
    txx_Sname VARCHAR(50) NOT NULL,  
    txx_Sex CHAR(2),  
    txx_Age INT,  
    txx_RegionID CHAR(4),  
    txx_DeptNo CHAR(4),  
    txx_ClassID CHAR(6),  
    txx_StartDate DATE NOT NULL,  
    txx_CreditHours INT DEFAULT 0,  
    FOREIGN KEY (txx_RegionID) REFERENCES Tianxx_Regions(txx_RegionID),  
    FOREIGN KEY (txx_DeptNo) REFERENCES Tianxx_Depts(txx_DeptNo),  
    FOREIGN KEY (txx_ClassID) REFERENCES Tianxx_Classes(txx_ClassID)  
);
```

◆ 课程表(不再含教师、班级、学期字段)(Tianxx_Courses)

存储课程信息，主键为课程编号。

```
CREATE TABLE Tianxx_Courses (  
    txx_Cno CHAR(6) PRIMARY KEY,  
    txx_Cname VARCHAR(100) NOT NULL,  
    txx_CreditHour INT NOT NULL,  
    txx_Hour INT,  
    txx_AssessType VARCHAR(10)  
);
```

◆ 教学表(Tianxx_Teaching)

建立课程、教师、班级与学期的多对多关系，主键为自增编号。

```
CREATE TABLE Tianxx_Teaching (  
    txx_TeachingID SERIAL PRIMARY KEY,  
    txx_Cno CHAR(6) NOT NULL,  
    txx_Tno CHAR(6) NOT NULL,
```

```

txx_ClassID CHAR(6) NOT NULL,
txx_Term VARCHAR(20) NOT NULL,
FOREIGN KEY (txx_Cno) REFERENCES Tianxx_Courses(txx_Cno),
FOREIGN KEY (txx_Tno) REFERENCES Tianxx_Teachers(txx_Tno),
FOREIGN KEY (txx_ClassID) REFERENCES Tianxx_Classes(txx_ClassID),
UNIQUE (txx_Cno, txx_Tno, txx_ClassID, txx_Term)
);

```

◆ 成绩表，引用教学ID (Tianxx_Reports)

存储学生成绩，联合主键由学号和教学安排编号组成。

```

CREATE TABLE Tianxx_Reports (
    txx_Sno CHAR(8) NOT NULL,
    txx_TeachingID INT NOT NULL,
    txx_Grade DECIMAL(5, 2),
    PRIMARY KEY (txx_Sno, txx_TeachingID),
    FOREIGN KEY (txx_Sno) REFERENCES Tianxx_Students(txx_Sno),
    FOREIGN KEY (txx_TeachingID) REFERENCES Tianxx_Teaching(txx_TeachingID)
);

```

◆ 用户表(Tianxx_users)

存储系统用户信息，包含登录账号与权限。

```

CREATE TABLE Tianxx_users (
    txx_id SERIAL PRIMARY KEY,
    txx_username VARCHAR(50) UNIQUE NOT NULL,
    txx_password VARCHAR(100) NOT NULL,
    txx_role VARCHAR(20) NOT NULL, -- admin / teacher / student / staff
    txx_name VARCHAR(50) NOT NULL, -- 显示名
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

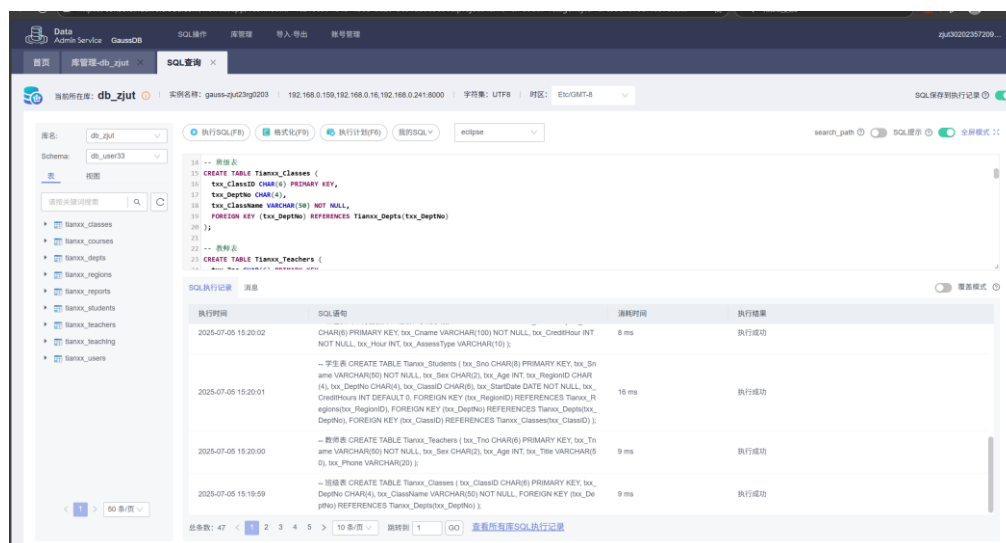


图15数据的插入

5.2 视图的建立

◆ 学生年度平均成绩视图

该视图统计每个学生在各学年的平均成绩，按院系分组，便于分析不同院系学生的学习情况。

```
CREATE OR REPLACE VIEW v_student_yearly_avg_scores AS
SELECT
    d.txx_DeptNo      AS dept_no,
    d.txx_DeptName    AS dept_name,
    s.txx_Sno         AS sno,
    s.txx_Sname       AS sname,
    SUBSTRING(t.txx_Term FROM 1 FOR 9) AS year,  -- 提取 2023-2024
    ROUND(AVG(r.txx_Grade), 2) AS avg_grade
FROM Tianxx_Reports r
JOIN Tianxx_Students s ON r.txx_Sno = s.txx_Sno
JOIN Tianxx_Teaching t ON r.txx_TeachingID = t.txx_TeachingID
JOIN Tianxx_Courses c ON t.txx_Cno = c.txx_Cno
JOIN Tianxx_Depts d ON s.txx_DeptNo = d.txx_DeptNo
GROUP BY d.txx_DeptNo, d.txx_DeptName, s.txx_Sno, s.txx_Sname, year
ORDER BY year DESC, avg_grade DESC;
```

◆ 教师课程学生成绩视图

汇总教师所授课程及对应学生成绩与联系方式。

```
CREATE OR REPLACE VIEW vw_teacher_course_student_grades AS
SELECT
    t.txx_Tno AS teacher_no,
    t.txx_Tname AS teacher_name,
    te.txx_TeachingID AS teaching_id,
    c.txx_Cno AS course_no,
    c.txx_Cname AS course_name,
    c.txx_CreditHour AS course_credits,
    c.txx_AssessType AS assess_type,
    cl.txx_ClassID AS class_id,
    cl.txx_ClassName AS class_name,
    te.txx_Term AS term,
    s.txx_Sno AS student_no,
    s.txx_Sname AS student_name,
    s.txx_Sex AS student_sex,
    s.txx_Phone AS student_phone,
    s.txx_Address AS student_address,
    r.txx_Grade AS grade
FROM Tianxx_Teachers t
JOIN Tianxx_Teaching te ON t.txx_Tno = te.txx_Tno
JOIN Tianxx_Courses c ON te.txx_Cno = c.txx_Cno
```

```

JOIN Tianxx_Classes cl ON te.txx_ClassID = cl.txx_ClassID
JOIN Tianxx_Students s ON s.txx_ClassID = cl.txx_ClassID
LEFT JOIN Tianxx_Reports r ON r.txx_Sno = s.txx_Sno AND r.txx_TeachingID =
te.txx_TeachingID;

```

◆ 学生课程成绩视图

查询学生所有课程及成绩（含未评分课程）。

```

CREATE OR REPLACE VIEW v_student_course_grades AS
SELECT
    s.txx_Sno AS student_no,
    s.txx_Sname AS student_name,
    c.txx_Cno AS course_no,
    c.txx_Cname AS course_name,
    t.txx_Term AS term,
    r.txx_Grade AS grade
FROM Tianxx_Students s
JOIN Tianxx_Classes cl ON s.txx_ClassID = cl.txx_ClassID
JOIN Tianxx_Teaching t ON cl.txx_ClassID = t.txx_ClassID
JOIN Tianxx_Courses c ON t.txx_Cno = c.txx_Cno
LEFT JOIN Tianxx_Reports r ON r.txx_Sno = s.txx_Sno AND r.txx_TeachingID =
t.txx_TeachingID
WHERE s.txx_Sno = 'S000001';

```

◆ 课程学期平均成绩视图

```

CREATE OR REPLACE VIEW v_course_term_avg_grade AS
SELECT
    c.txx_Cno AS course_no,
    c.txx_Cname AS course_name,
    t.txx_Term AS term,
    ROUND(AVG(r.txx_Grade), 2) AS avg_grade
FROM Tianxx_Courses c
JOIN Tianxx_Teaching t ON c.txx_Cno = t.txx_Cno
JOIN Tianxx_Reports r ON t.txx_TeachingID = r.txx_TeachingID
GROUP BY c.txx_Cno, c.txx_Cname, t.txx_Term
ORDER BY t.txx_Term DESC, avg_grade DESC;

```

◆ 教师课程学生人数统计视图

```

CREATE OR REPLACE VIEW v_teacher_courses_student_count AS
SELECT
    t.txx_Tno AS teacher_no,
    t.txx_Tname AS teacher_name,
    c.txx_Cno AS course_no,
    c.txx_Cname AS course_name,
    te.txx_Term AS term,

```

```

cl.txx_ClassID AS class_id,
cl.txx_ClassName AS class_name,
COUNT(s.txx_Sno) AS student_count
FROM Tianxx_Teachers t
JOIN Tianxx_Teaching te ON t.txx_Tno = te.txx_Tno
JOIN Tianxx_Courses c ON te.txx_Cno = c.txx_Cno
JOIN Tianxx_Classes cl ON te.txx_ClassID = cl.txx_ClassID
LEFT JOIN Tianxx_Students s ON s.txx_ClassID = cl.txx_ClassID
GROUP BY t.txx_Tno, t.txx_Tname, c.txx_Cno, c.txx_Cname, te.txx_Term,
cl.txx_ClassID, cl.txx_ClassName;

```

◆ 学生学期累计学分视图

```

CREATE OR REPLACE VIEW v_student_term_credits AS
SELECT
    s.txx_Sno AS student_no,
    s.txx_Sname AS student_name,
    SUBSTRING(t.txx_Term FROM 1 FOR 9) AS term_year,
    SUM(c.txx_CreditHour) AS total_credits
FROM Tianxx_Students s
JOIN Tianxx_Classes cl ON s.txx_ClassID = cl.txx_ClassID
JOIN Tianxx_Teaching t ON cl.txx_ClassID = t.txx_ClassID
JOIN Tianxx_Courses c ON t.txx_Cno = c.txx_Cno
JOIN Tianxx_Reports r ON r.txx_Sno = s.txx_Sno AND r.txx_TeachingID = t.txx_TeachingID
WHERE r.txx_Grade >= 60 -- 及格学分统计
GROUP BY s.txx_Sno, s.txx_Sname, term_year
ORDER BY s.txx_Sno, term_year DESC;

```

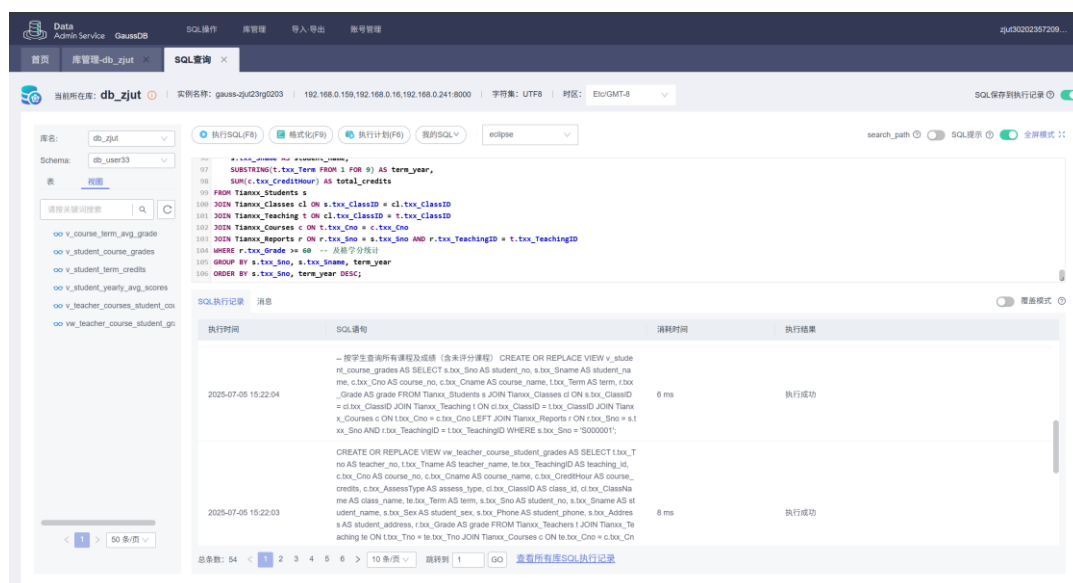


图 16 视图的建立

5.3 索引的建立

为了提高数据库的查询性能和数据访问效率，针对系统中常用的查询条件和关联字段，设计并建立了如下索引：

◆ 地区表(Tianxx_Regions)

--主键索引：已自动建立在字段 txx_RegionID 上。

◆ 系表 (Tianxx_Depts)

--主键索引：已自动建立在字段txx_RegionID上。

◆ 班级表(Tianxx_Classes)

--主键索引：已自动建立在字段 txx_ClassID 上。

--外键索引：加快系部过滤效率，建议：

```
CREATE INDEX idx_classes_deptno ON
Tianxx_Classes(txx_DeptNo);
```

◆ 学生表(Tianxx_Students)

--主键索引：已自动建立在字段 txx_Sno 上。

--外键索引：提升关联查询效率，建议：

```
CREATE INDEX idx_students_regionid ON
Tianxx_Students(txx_RegionID);
CREATE INDEX idx_students_deptno ON
Tianxx_Students(txx_DeptNo);
CREATE INDEX idx_students_classid ON
Tianxx_Students(txx_ClassID);
```

◆ 教学安排表(Tianxx_Teaching)

--主键索引：字段 txx_TeachingID 上。

--唯一索引：已自动建立在组合字段：

```
(txx_Cno, txx_Tno, txx_CClassID, txx_Term)
```

--组合查询优化索引：教师 + 班级 + 学期组合筛选优化：

```
CREATE INDEX idx_teaching_teacher_class_term
ON Tianxx_Teaching(txx_Tno, txx_ClassID,
txx_Term)
```

◆ 成绩表(Tianxx_Reports)

--主键索引：自动建立在联合主键 (txx_Sno, txx_TeachingID)。

--辅助查询索引：提高按学号或教学编号查成绩效率：

```
CREATE INDEX idx_reports_student ON
Tianxx_Reports(txx_Sno);
CREATE INDEX idx_reports_teaching ON
Tianxx_Reports(txx_TeachingID);
```

◆ 用户表(Tianxx_Users)

--主键索引：字段 `txx_id` 上。

--唯一索引：用户名字段 `txx_username` 的唯一性自动建立索引。

通过上述索引设计，系统在执行复杂查询时可以避免全表扫描，有效提升数据检索、统计分析与界面响应速度，增强整体性能。

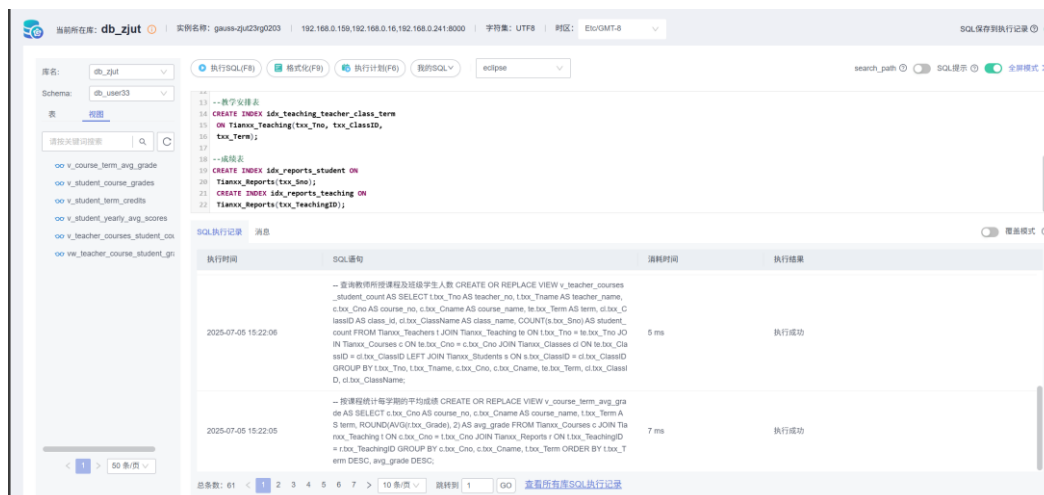


图 17 索引的建立

5.4 触发器建立

◆ 成绩变动后自动更新学生已修学分

功能说明：当成绩表中的学生成绩被插入或更新时，触发器自动计算该学生所有及格课程的学分总和，并更新学生表中的累计学分字段，确保学分数据的准确和同步。

```
CREATE OR REPLACE FUNCTION update_student_credits()
RETURNS TRIGGER AS $$
DECLARE
    total_credits INT;
BEGIN
    -- 计算该学生所有及格课程学分总和
    SELECT COALESCE(SUM(c.txx_CreditHour), 0)
    INTO total_credits
    FROM Tianxx_Reports r
    JOIN Tianxx_Teaching t ON r.txx_TeachingID = t.txx_TeachingID
    JOIN Tianxx_Courses c ON t.txx_Cno = c.txx_Cno
    WHERE r.txx_Sno = NEW.txx_Sno AND r.txx_Grade >= 60;

    -- 更新学生的已修学分
    UPDATE Tianxx_Students
    SET txx_CreditHours = total_credits
    WHERE txx_Sno = NEW.txx_Sno;

    RETURN NEW;
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_update_credits_after_report_change
AFTER INSERT OR UPDATE OF txx_Grade ON Tianxx_Reports
FOR EACH ROW
EXECUTE PROCEDURE update_student_credits();
```

◆ 删除学生时自动删除相关成绩记录

功能说明：在删除学生记录之前，触发器自动删除该学生所有相关成绩记录，避免数据孤立，维护数据完整性。

```
CREATE OR REPLACE FUNCTION delete_student_related_records()
RETURNS TRIGGER AS $$
BEGIN
    -- 删除学生相关成绩
    DELETE FROM Tianxx_Reports WHERE txx_Sno = OLD.txx_Sno;

    -- 如果还有其它关联表需要删除，也写在这里

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_delete_student_cascade
BEFORE DELETE ON Tianxx_Students
FOR EACH ROW
EXECUTE PROCEDURE delete_student_related_records();
```

◆ 新增学生时自动创建用户账户

功能说明：新增学生记录时，自动在用户表中创建对应账号，用户名为学号，默认密码及学生角色，实现身份信息与系统登录账户的同步。

```
CREATE OR REPLACE FUNCTION create_student_user()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Tianxx_users (txx_username, txx_password, txx_role, txx_name)
    VALUES (
        NEW.txx_Sno,           -- 用户名为学号
        '123456',             -- 默认密码（建议后续修改）
        'student',             -- 身份角色
        NEW.txx_Sname         -- 显示姓名
    );
    RETURN NEW;
```

```

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_create_user_after_student_insert
AFTER INSERT ON Tianxx_Students
FOR EACH ROW
EXECUTE PROCEDURE create_student_user();

```

◆ 删除教师时自动级联删除相关教学及成绩记录

功能说明：删除教师记录之前，触发器自动删除该教师所有教学安排及相关成绩，防止数据孤立，保持数据库一致性。

```

CREATE OR REPLACE FUNCTION delete_teacher_related_records()
RETURNS TRIGGER AS $$
DECLARE
    teaching_ids INT[];
    delete_count INT;
BEGIN
    SELECT ARRAY_AGG(txx_TeachingID)
    INTO teaching_ids
    FROM Tianxx_Teaching
    WHERE txx_Tno = OLD.txx_Tno;
    IF teaching_ids IS NULL THEN
        RAISE NOTICE '教师 % 无教学记录，直接删除', OLD.txx_Tno;
        RETURN OLD;
    END IF;
    SELECT COUNT(*) INTO delete_count
    FROM Tianxx_Reports
    WHERE txx_TeachingID = ANY(teaching_ids);
    RAISE NOTICE '正在删除教师 % 的 % 条成绩记录和 % 条教学记录',
        OLD.txx_Tno,
        delete_count,
        array_length(teaching_ids, 1);
    DELETE FROM Tianxx_Reports
    WHERE txx_TeachingID = ANY(teaching_ids);
    DELETE FROM Tianxx_Teaching
    WHERE txx_Tno = OLD.txx_Tno;
    RAISE NOTICE '教师 % 的相关记录已删除完毕', OLD.txx_Tno;
    RETURN OLD;
EXCEPTION
    WHEN OTHERS THEN
        RAISE EXCEPTION '删除教师 % 相关记录时出错：%', OLD.txx_Tno,
        SQLERRM;
        RETURN NULL; -- 这将阻止删除操作
END;

```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_delete_teacher_cascade
BEFORE DELETE ON Tianxx_Teachers
FOR EACH ROW
EXECUTE PROCEDURE delete_teacher_related_records();
```

整体上, 这些触发器确保了学生和教师相关数据的自动维护, 减少人工干预, 提升系统的可靠性和数据安全性。

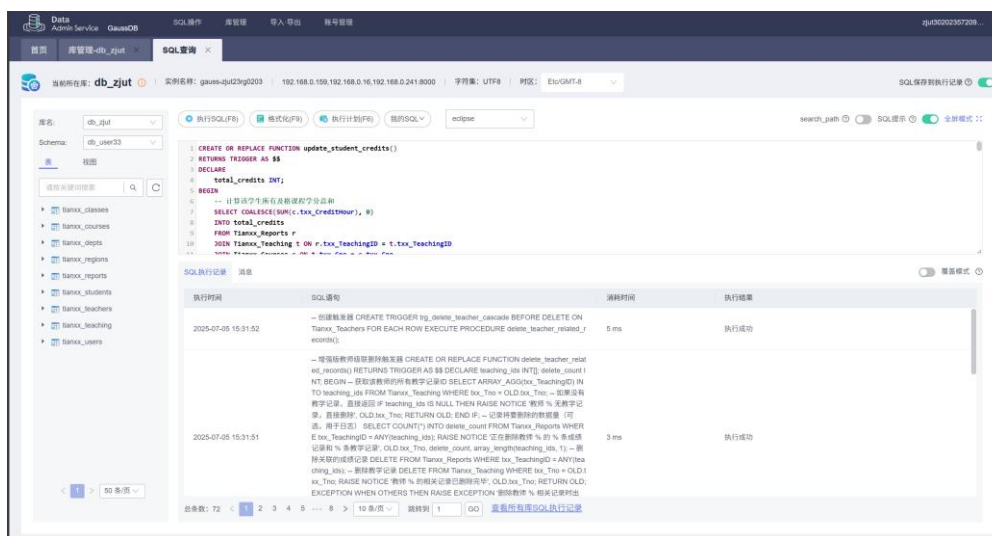


图 18 建立触发器

5.5 建存储过程

◆ 学生学分统计存储过程

```
CREATE OR REPLACE PROCEDURE sp_calculate_student_credits(
    IN student_no CHAR(8),
    OUT total_credits INT,
    OUT remaining_credits INT
)
AS
DECLARE
    required_credits INT := 160; -- 假设毕业需要160学分
BEGIN
    -- 计算已获得的总学分（及格课程）
    SELECT COALESCE(SUM(c.txx_CreditHour), 0)
    INTO total_credits
    FROM Tianxx_Reports r
    JOIN Tianxx_Teaching t ON r.txx_TeachingID = t.txx_TeachingID
    JOIN Tianxx_Courses c ON t.txx_Cno = c.txx_Cno
    WHERE r.txx_Sno = student_no
    AND r.txx_Grade >= 60;

    -- 计算剩余所需学分
```

```

remaining_credits := GREATEST(required_credits - total_credits, 0);

-- 更新学生表中的学分字段
UPDATE Tianxx_Students
SET txx_CreditHours = total_credits
WHERE txx_Sno = student_no;
END;
```

◆ 教师课程管理存储过程

```

CREATE OR REPLACE PROCEDURE sp_teacher_course_management(
    IN teacher_no CHAR(6),
    IN term VARCHAR(20),
    OUT course_count INT,
    OUT student_count INT
)
AS
BEGIN
    -- 统计教师某学期教授的课程数量
    SELECT COUNT(DISTINCT t.txx_Cno)
    INTO course_count
    FROM Tianxx_Teaching t
    WHERE t.txx_Tno = teacher_no
        AND t.txx_Term = term;

    -- 统计教师某学期教授的学生总数
    SELECT COUNT(DISTINCT s.txx_Sno)
    INTO student_count
    FROM Tianxx_Teaching t
    JOIN Tianxx_Classes c ON t.txx_ClassID = c.txx_ClassID
    JOIN Tianxx_Students s ON s.txx_ClassID = c.txx_ClassID
    WHERE t.txx_Tno = teacher_no
        AND t.txx_Term = term;

    -- 使用GaussDB兼容的输出方式
    RAISE NOTICE '教师 % 在 % 学期教授 % 门课程，共 % 名学生',
        teacher_no, term, course_count, student_count;
END;
```

◆ 学生成绩分析存储过程

- 主键索引：已自动建立在字段 txx_ClassID 上。
- 外键索引：加快系部过滤效率，建议：

```

CREATE OR REPLACE PROCEDURE sp_analyze_student_performance(
    IN student_no CHAR(8),
    OUT avg_grade DECIMAL(5,2),
```

```
OUT highest_grade DECIMAL (5, 2),
OUT lowest_grade DECIMAL (5, 2),
OUT passed_courses INT,
OUT failed_courses INT
)
AS
BEGIN
    -- 计算平均成绩（排除NULL值）
    SELECT ROUND (AVG (txx_Grade), 2)
    INTO avg_grade
    FROM Tianxx_Reports
    WHERE txx_Sno = student_no
        AND txx_Grade IS NOT NULL;

    -- 获取最高分（包含所有成绩）
    SELECT MAX (txx_Grade)
    INTO highest_grade
    FROM Tianxx_Reports
    WHERE txx_Sno = student_no;

    -- 获取最低分（排除NULL和负值）
    SELECT MIN (txx_Grade)
    INTO lowest_grade
    FROM Tianxx_Reports
    WHERE txx_Sno = student_no
        AND txx_Grade >= 0;

    -- 统计及格课程数（≥60分）
    SELECT COUNT (*)
    INTO passed_courses
    FROM Tianxx_Reports
    WHERE txx_Sno = student_no
        AND txx_Grade >= 60;

    -- 统计不及格课程数（0≤分数<60）
    SELECT COUNT (*)
    INTO failed_courses
    FROM Tianxx_Reports
    WHERE txx_Sno = student_no
        AND txx_Grade < 60
        AND txx_Grade >= 0;

    RAISE NOTICE '学号 % 的成绩分析：平均分 %, 最高分 %, 最低分 %, 及格课
程 %, 不及格课程 %',
```

```

student_no,
COALESCE (avg_grade, 0),
COALESCE (highest_grade, 0),
COALESCE (lowest_grade, 0),
COALESCE (passed_courses, 0),
COALESCE (failed_courses, 0);

END;

```

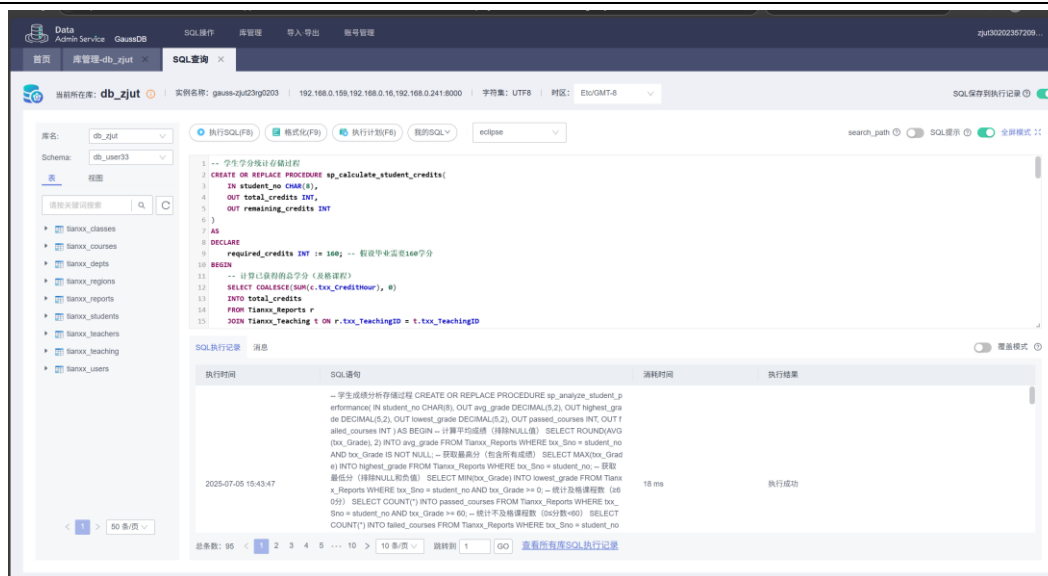


图 19 建立存储过程

6、高校教务管理系统开发与运行

6.1 开发平台和开发环境介绍

本系统采用前后端分离的架构,结合现代主流的开发框架与工具,确保系统的可扩展性、易维护性及良好的用户交互体验。开发环境如下所述:

- 开发操作系统: 全过程在 windows11 操作系统下完成,具有良好的兼容性与稳定性,适合进行前后端联合开发。
- 后端开发环境: 使用 Python 3.8 编写后端接口,主要框架为 FastAPI,具有异步支持、自动生成文档等特性,便于快速构建 FastAPI 接口。使用 PyCharm 作为主要开发 IDE,具备完善的代码提示、调试与版本控制支持。
- 数据库系统: 后端数据库采用 GaussDB,具有事务支持、约束完整性强、扩展性好等优势,适合关系型数据建模。
- 前端开发环境: 前端采用 Vue.js 框架进行开发,结合 Element UI 组件库实现丰富的页面交互与组件样式;页面结构和样式主要基于 HTML5 与 CSS3。
- 项目结构:
 - 前端项目: 以 Vue 单页应用方式构建,使用 npm 管理依赖。
 - 后端项目: 以模块化 Python 项目组织,使用 FastAPI 运行服务。
- 其他工具:
 - Git: 用于版本控制与代码托管。

- Postman: 用于调试后端接口。
- Terminal + curl: 用于快速测试与部署指令执行。

通过上述开发平台与工具的结合,实现了从数据库设计、后端服务构建、前端界面展示到系统联调的一体化开发流程,提升了开发效率与系统质量。

6.2 数据准备

准备了以下表的数据:

- 地区表(Tianxx_Regions): 包含 5 个地区记录
 - 地区 ID (如'R001')
 - 地区名称 (如'浙江')
- 系别表(Tianxx_Depts): 包含 4 个系别记录
 - 系别 ID (如'D001')
 - 系别名称 (如'计算机')
- 班级表(Tianxx_Classes): 包含 8 个班级记录(每个系别 2 个班级)
 - 班级 ID (如'C001')
 - 所属系别 ID
 - 班级名称 (如'计算机 1 班')
- 教师表(Tianxx_Teachers): 包含 10 位教师记录
 - 教师 ID (如'T001')
 - 姓名、性别、年龄、职称和联系电话
- 学生表(Tianxx_Students): 包含 100 位学生记录
 - 学号 (如'S000001')
 - 姓名、性别、年龄、所属地区 ID、所属系别 ID、班级 ID、入学日期
- 课程表(Tianxx_Courses): 包含 20 门课程记录
 - 课程 ID (如'K001')
 - 课程名称、学分、学时和考核方式(考试/考查)
- 教学安排表(Tianxx_Teaching): 包含 50 条教学安排记录
 - 教学 ID、课程 ID、教师 ID、班级 ID 和学期(如'2023-2024-1')
- 成绩表(Tianxx_Reports): 包含大量成绩记录
 - 学号、教学 ID 和成绩

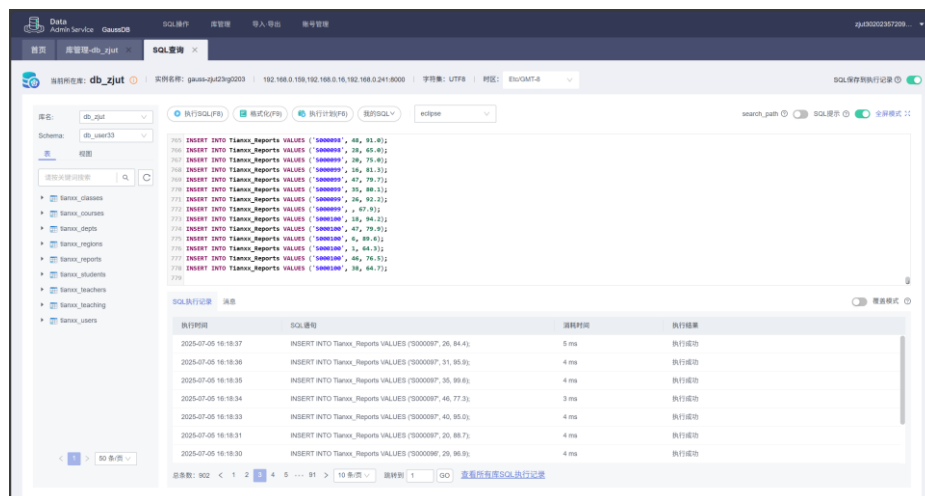


图 20 数据插入

6.3 前台界面与后台数据库连接说明

本系统采用前后端分离架构，前端界面通过调用后端提供的 API 接口与数据库进行交互，确保了系统的高内聚低耦合结构设计。具体说明如下：

- 通信方式：前端使用 Axios 向后端发起 HTTP 请求，主要采用 GET、POST、PUT、DELETE 等方法，实现数据的查询、添加、修改和删除。
- 数据格式：前后端之间的数据传输采用 JSON（JavaScript Object Notation）格式，便于解析与处理。
- 接口设计：后端使用 FastAPI 框架编写 FastAPI 接口，每个功能模块对应一组接口。

接口路径	方法	功能说明
/dashboard/stats	GET	获取学生、教师、课程、院系数量总览
/dashboard/charts/students_by_dept	GET	获取按院系分组的学生人数
/dashboard/charts/major	GET	获取专业分布数据（复用上一个接口）
/dashboard/charts/grade_distribution	GET	获取成绩分布区间统计
/dashboard/charts/hometown_distribution	GET	获取学生源地分布
/students	GET	获取学生列表（可分页）
/students_add	POST	添加学生信息
/students_import	POST	批量导入学生
/students_edit	POST	修改学生信息
/students_delete/{sno}	DELETE	删除指定学号的学生
/students/export	GET	导出学生数据
/students_dashboard/{sno}	GET	获取学生仪表盘信息
/students_info/{sno}	GET	获取学生个人信息详情
/students/{sno}/update_profile	POST	学生更新个人基本信息（地址、电话）
/students/{sno}/select_course	POST	学生选课操作
/students/{sno}/selected_courses	GET	获取学生已选课程列表
/students/{sno}/grades	GET	获取学生成绩记录
/students_courses/{sno}	GET	获取学生所学课

		程
/students_grades/{sno}	GET	获取学生课程成绩
/students/detail_with_grades	GET	获取学生详情及成绩信息
/teachers	GET	获取教师列表
/teachers_add	POST	添加教师信息
/teachers_import	POST	批量导入教师
/teachers_delete/{tno}	DELETE	删除教师信息
/teachers_edit	POST	编辑教师信息
/teachers/export	GET	导出教师信息
/teacher/{teacher_no}/dashboard	GET	获取教师个人主页统计数据
/teachers/{tno}/update_profile	POST	教师更新个人信息
/teachers/{tno}/courses	GET	获取教师授课列表
/teacher_courses	GET	教师授课查询
/teachers/{tno}/course_students	GET	获取教师课程下的学生
/teachers/{tno}/submit_grade	POST	教师提交学生成绩
/teachers/{tno}/course_students_scores	GET	查询课程成绩
/courses	GET	获取课程列表
/courses_add	POST	添加课程
/courses_edit	POST	编辑课程
/courses/{cno}	PUT	更新课程信息
/courses_delete/{cno}	DELETE	删除课程
/course_teaching_classes	GET	获取课程对应班级的授课安排
/courses/avg_grades	GET	查询课程平均成绩
/courses/export	GET	导出课程数据
/courses/available	GET	查询可选课程列表
/departments	GET	获取院系列表（重复注册）
/departments_get	GET	获取院系列表
/departments_add	POST	添加院系
/departments_update	PUT	更新院系
/departments_delete/{dept_no}	DELETE	删除院系
/departments/{dept_no}/classes	GET	获取院系对应的班级列表

/grades/year_summary	GET	获取学生年度成绩统计
/grades/add_by_name	POST	通过学生名添加成绩
/grades/update_by_name	POST	更新成绩
/grades/earned_credits	GET	获取学生已修学分
/grades/export	GET	导出成绩表
/login	POST	用户登录接口
/langchain_qa	POST	AI 问答接口（实验性质）
/docs, /redoc, /openapi.json	GET	Swagger 与 Redoc 文档自动生成接口

◆ 数据库连接：

```
import psycopg2
# 数据库连接信息
conn_info = {
    "dbname": "db_zjut",
    "user": "db_user33",
    "password": "db_user33@123",
    "host": "110.41.127.77",
    "port": "8000"
}

def get_conn():
    return psycopg2.connect(**conn_info)
```

◆ 调用示例：以学生信息查询为例，前端通过如下语句调用接口：

```
axios.get(`/students/${studentId}`) .then(response => {
  this.studentInfo = response.data; });
```

后端接口处理如下请求：

```
@app.get("/students/{student_id}") def get_student(student_id: str):
    conn = get_conn() cur = conn.cursor()
    cur.execute("SELECT *FROM Tianxx_Students WHERE txx_Sno=%s",
    (student_id,))
    row = cur.fetchone() ...
```

◆ 权限控制：系统通过用户登录接口校验身份，依据用户角色（管理员、教师、学生）返回相应功能权限，前端从本地存储中读取用户信息以决定界面展示内容与可访问功能。

6.4 应用系统的开发

6.4.1 系统各功能设计

本系统基于 FastAPI 框架开发，按照典型的前后端分离架构进行设计，后端主要提供统一的 RESTful API 接口服务。系统功能涵盖用户登录认证、数据统计可视化、学生与教师管理、课程与成绩管理、院系维护等多个子模块，具体设计如下：

- 用户认证模块：提供统一的用户身份验证服务，支持教师与学生角色的登录操作。认证成功后生成访问令牌，用于保护用户信息和接口调用的安全性。此外，系统集成了 API 文档自动生成工具，便于前后端协同开发和接口测试。
- 数据统计与分析模块：在用户登录后可访问的仪表盘，系统展示了学生、教师、课程和院系的总览信息，并通过图表形式展示学生不同院系、专业、成绩分布、生源地等维度的统计结果。这一模块便于管理员全面掌握系统运行情况及数据分布特征。
- 学生信息管理模块：支持管理员对学生信息进行全生命周期管理，包括学生的新增、修改、删除、导入、导出等功能。同时，每位学生还可以查看并更新自己的基本信息，如联系方式和地址。此外，学生可在线选课，查看所选课程及对应的成绩信息。
- 教师信息管理模块：管理员可完成教师的注册、信息维护、导入导出等操作。系统支持为每位教师展示其个人主页，展示其所授课程与教学数据。教师也可自行维护其个人资料，并查看或管理授课班级的学生信息及成绩录入情况。
- 课程管理模块：包括课程的添加、编辑、删除、导出、授课安排管理等功能。系统支持按班级安排课程教学任务，教师与学生可分别查看各自相关的课程信息。此外，课程模块也支持对课程平均成绩等统计数据的查询。
- 成绩管理模块：该模块提供对学生成绩的增删改查操作，支持按学生、课程等多维度进行查询与统计。教师可提交学生成绩，系统可自动统计学生年度成绩、计算所修学分并导出成绩表，辅助教学数据分析与学生评估。
- 院系与班级管理模块：系统允许管理员维护院系及其下属班级的信息。各模块可通过院系编号进行数据关联，用于学生分类展示、按院系统计等功能，保证系统结构的层次性与可扩展性。

6.4.2 应用系统结构

前端部分采用传统的 HTML+CSS+JavaScript 技术栈，包含多个功能页面（如登录页、学生/教师管理页），并通过 api.js 与后端交互，charts.js 实现数据可视化，utils.js 提供通用工具函数，整体结构清晰，便于维护和扩展。

后端部分基于 Python 开发，采用功能模块化设计，包括用户认证 (login.py)、学生管理 (student_manager.py)、教师管理 (teacher_manager.py)、课程管理 (course_manage.py) 等核心业务模块，并通过 connect.py 与数据库交互。数据库独立存储在 D:\database_txx 目录，确保数据安全性和可移植性。

采用前后端分离的设计模式，构建了一个模块化、可扩展的教育管理平台。前端基于 HTML+CSS+JavaScript 实现用户交互，并通过 API 与后端通信；后端采用 Python 编写，按功能模块划分，结合独立数据库存储，确保业务逻辑清晰、数据安全可靠。整体架构层次分明，便于维护和功能扩展，适用于学校、培训机构等场景，满足用户管理、课程管理、数据可视化等核心需求，具有良好的可维护性和可伸缩性。

详细请见图 21.

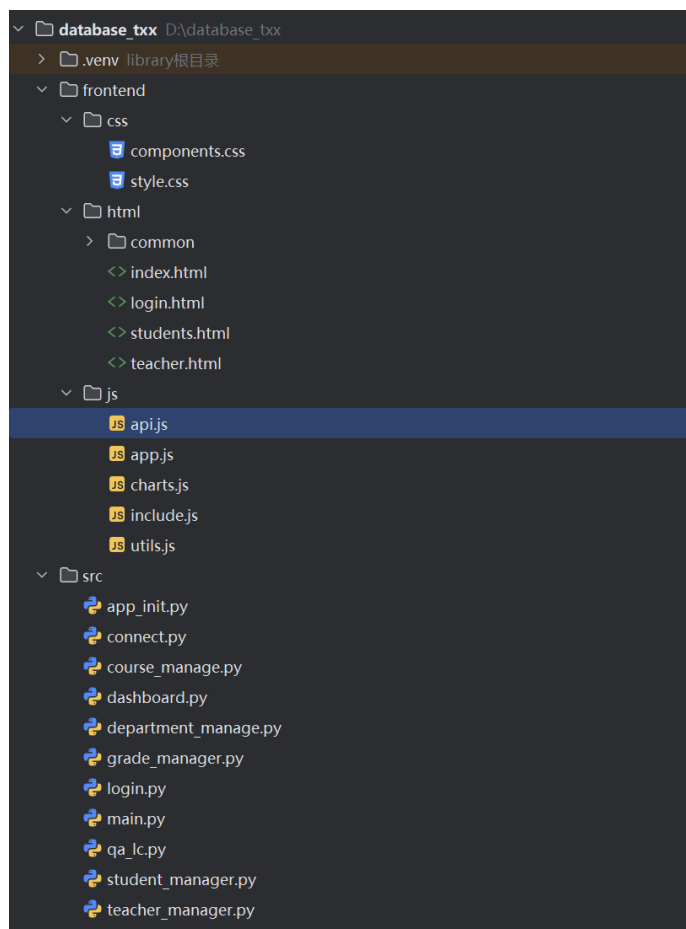


图 21 系统结构

6.5 系统界面展示

登录界面如图 22 所示。

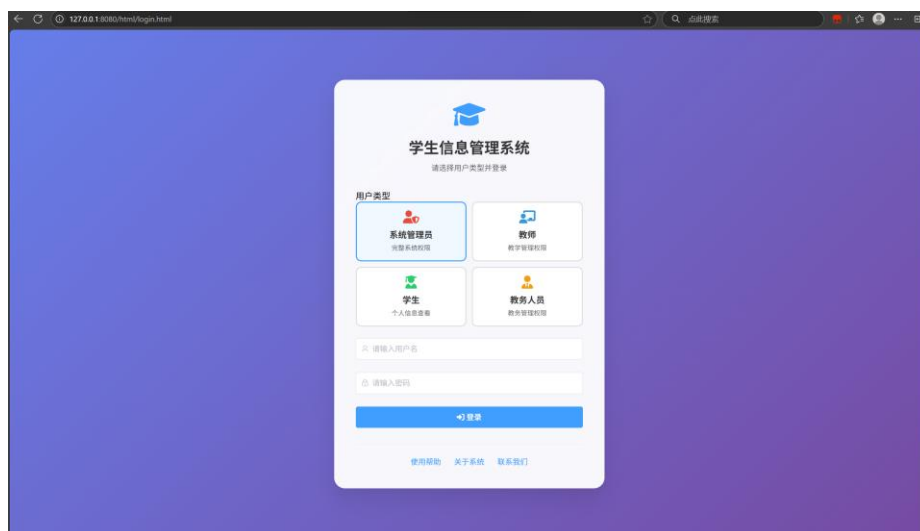


图 22 登录界面

6.5.1 系统管理员界面&教务管理界面

管理员主界面如图 23 所示，学生管理界面如图 24 所示，学生添加界面如图 25 所

示。学生详细查看界面如图 26 所示，成绩查询界面如图 27 所示，专业管理界面如图 28 所示。



图 23 管理员主界面

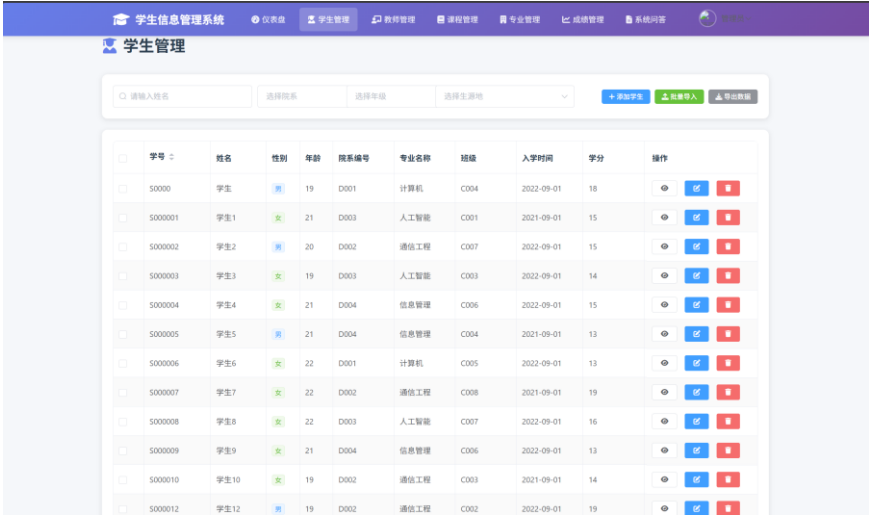


图 24 学生管理界面

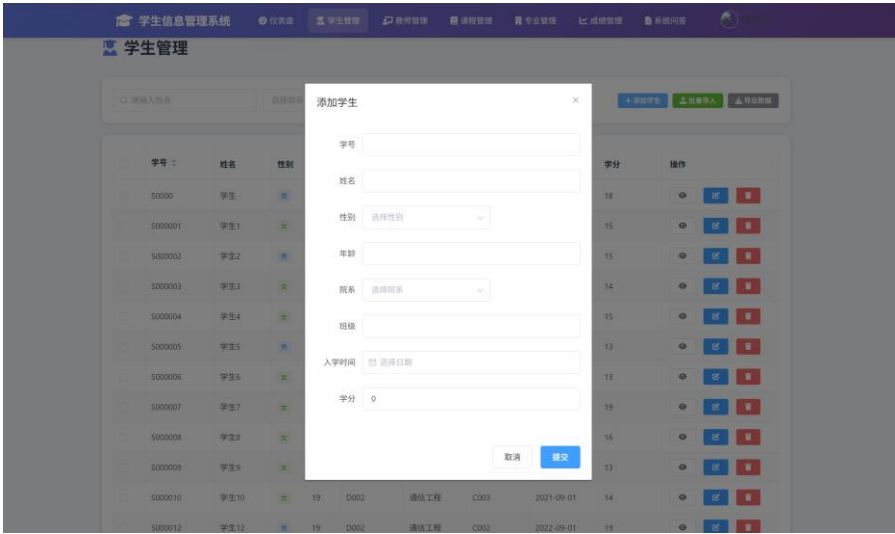


图 25 学生添加界面

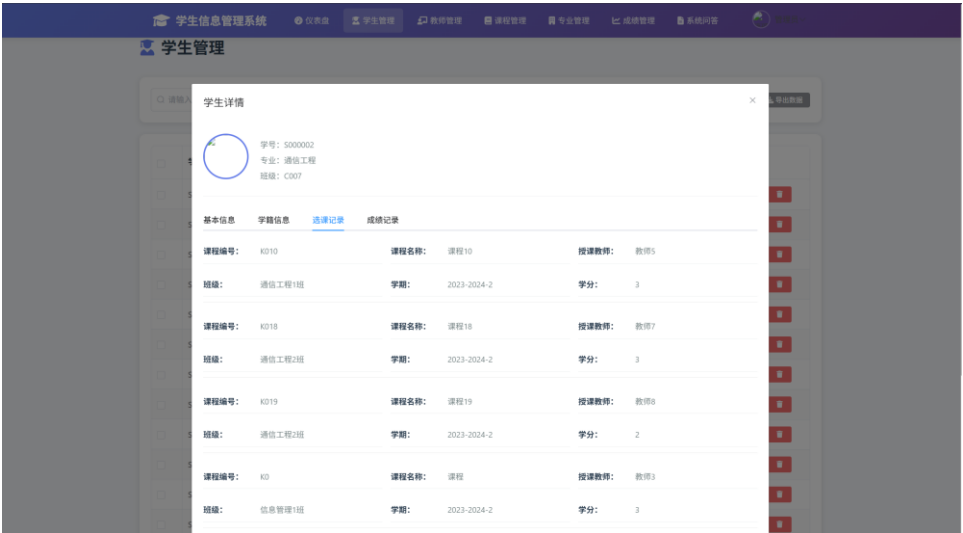


图 26 学生详细查看界面

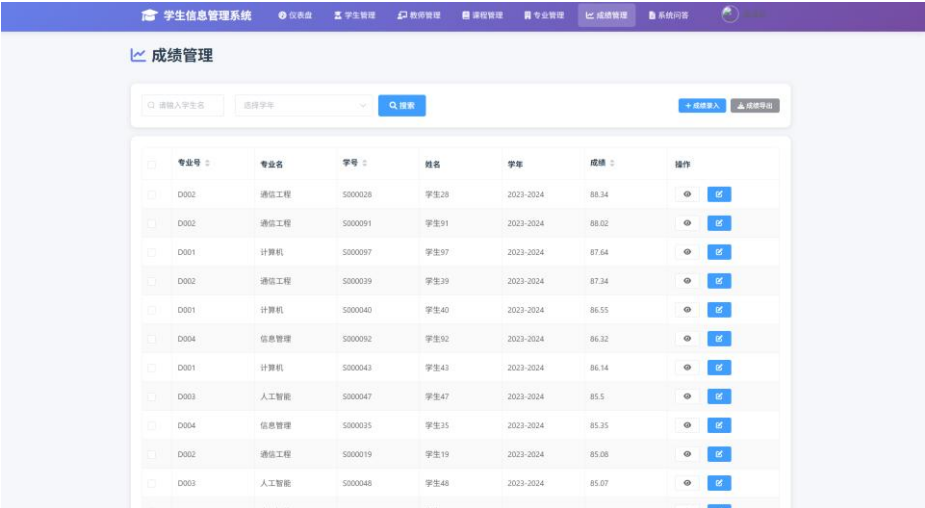


图 27 成绩查询界面

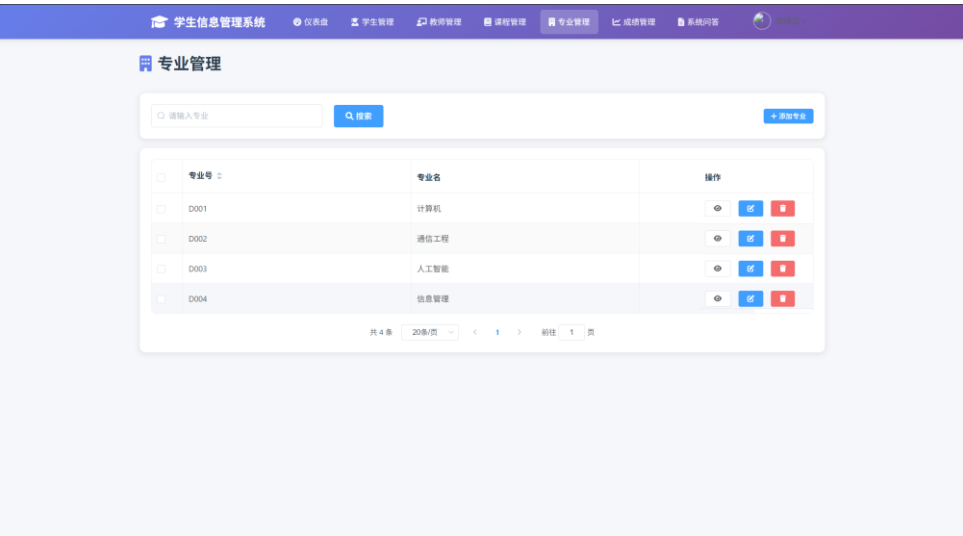


图 28 专业管理界面

Ps：课程管理界面、教师管理界面与学生管理界面类似，不做赘述。

6.5.2 教师界面

教师端主界面如图 29 所示，教师端成绩查询界面如图 30 所示，成绩录入界面如图 31 所示（可批量录入）。

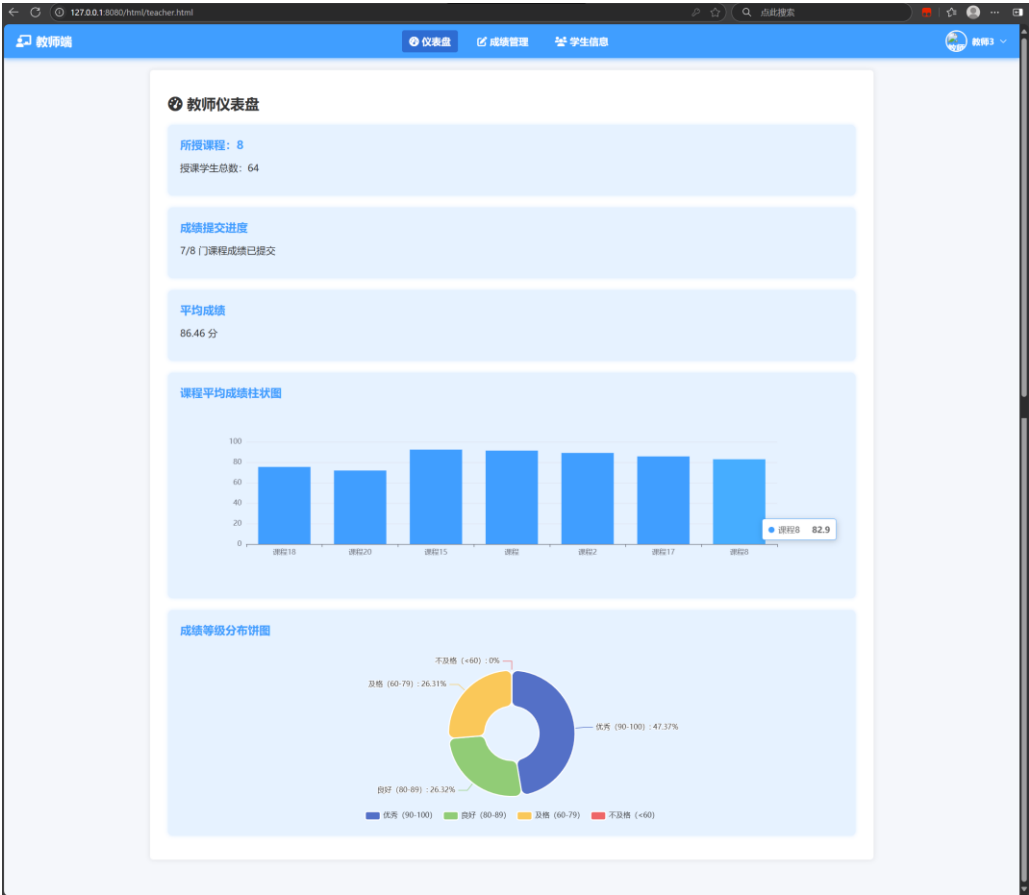


图 29 教师端主界面

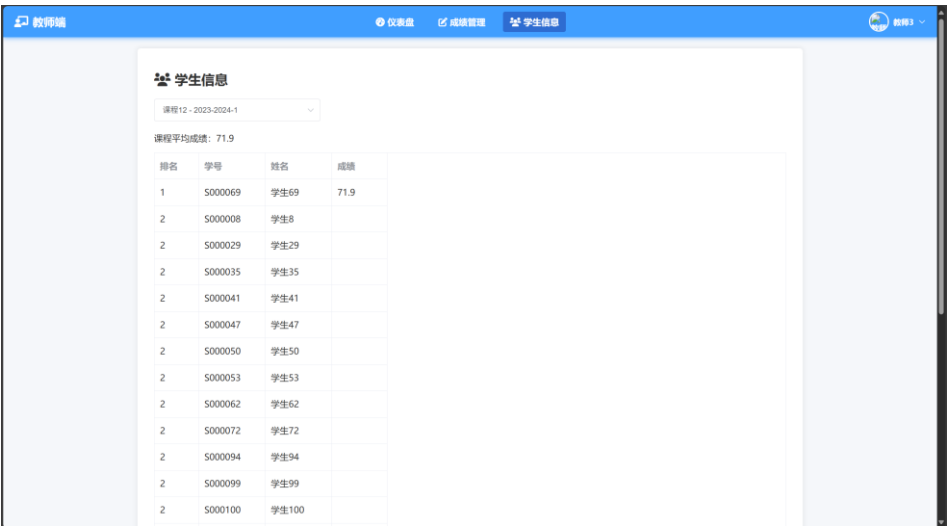


图 30 教师端成绩查询界面



图 31 成绩录入界面

6.5.3 学生界面

学生端主界面如图 32 所示，学生端个人信息修改界面如图 33 所示，学生成绩查询界面如图 34 所示，学生端选课界面如图 35 所示。



图 32 学生端主界面

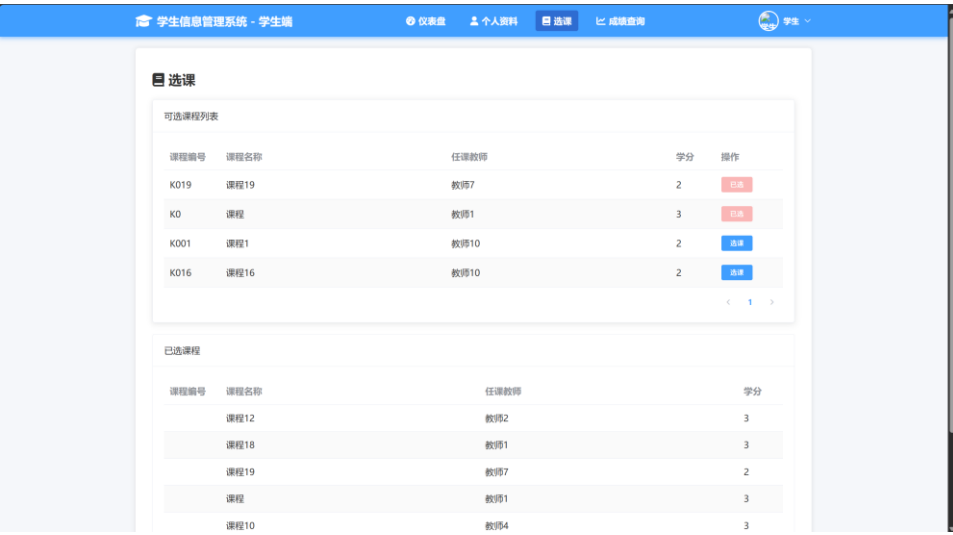
学号	姓名	手机号	地址
S000001	学生1		

图 33 学生端个人信息修改界面



学期	课程编号	课程名称	任课教师	成绩
2023-2024-2	K012	课程12	教师2	78.10
2023-2024-2	K018	课程18	教师1	81.20
2023-2024-2	K019	课程19	教师7	78.40
2023-2024-1	K0	课程	教师1	暂无成绩
2023-2024-1	K008	课程8	教师8	98.60
2023-2024-1	K010	课程10	教师4	75.80
2023-2024-1	K019	课程19	教师1	66.90

图 34 学生成绩查询界面



可选课程列表				
课程编号	课程名称	任课教师	学分	操作
K019	课程19	教师7	2	选课
K0	课程	教师1	3	选课
K001	课程1	教师10	2	选课
K016	课程16	教师10	2	选课

课程编号	课程名称	任课教师	学分
	课程12	教师2	3
	课程18	教师1	3
	课程19	教师7	2
	课程	教师1	3
	课程10	教师4	3

图 35 学生端选课界面

7、课程设计总结

7.1 遇到的问题和解决的办法

在本系统的设计与实现过程中，虽总体进展顺利，但也遇到了一些技术性和格式性的问题。通过查阅资料、调试代码与请教同伴，最终均得以解决。以下列举主要问题及对应的解决方案：

1. 接口命名与路径管理混乱：初期开发中接口路径存在命名不一致、重复注册等问题，导致部分请求无法正确响应。解决方法是统一接口路径命名规则（例如统一使用下划线连接）、梳理所有接口列表，并结合 FastAPI 的自动文档工具对每个接口进行注释和验证，确保接口唯一性与语义清晰。

2. 数据分页与前端展示对接复杂：为了提高接口性能和用户体验，大部分数据查询接口都设计为分页返回。在前端实现过程中，需要动态获取页码与总条数。为解决这一问题，

后端标准化返回结构体格式，统一返回 `total`、`items` 等字段，并在前端配合 Element UI 的分页组件完成联动。

3. 选课逻辑与成绩录入权限控制：教师录入成绩时需要确保学生确实选修了课程，避免越权操作。为此，在成绩提交接口中加入课程与学生关联关系校验机制，若学生未选课则返回错误提示，确保数据一致性和系统安全性。

7.2 系统设计的不足

本系统在设计与开发过程中尽力追求功能完整性与结构合理性，但由于项目周期有限、团队资源受限以及个人技术经验的局限，系统仍存在以下几个方面的不足：

1. 接口权限控制机制不够完善：当前系统虽然实现了基本的登录验证功能，但尚未对不同用户角色进行细粒度的权限划分。部分敏感接口（如删除操作、成绩管理）在后端缺乏严格的身份验证机制，存在越权访问的潜在风险。

2. 前端交互体验较为基础：由于前端开发时间受限，页面布局和交互效果相对简洁，未实现高级功能如表格排序、筛选、实时搜索、弹窗校验等，用户体验仍有提升空间。

3. 缺乏完整的异常处理与错误提示：当前接口返回错误信息较为简单，未能针对不同类型的错误（如参数缺失、数据库操作失败、权限异常）给予详细反馈，可能对用户造成困扰，也增加了调试和维护的难度。

4. 数据库结构扩展性不足：虽然已设计多张实体表并考虑基本的范式要求，但在多对多关系建模（如学生选课）方面仍存在字段冗余、关联不清的问题，未来若扩展如课程冲突检测、时间安排优化等高级功能将面临一定的结构调整成本。

5. 缺乏系统日志与操作追踪机制：当前系统未集成日志记录功能，管理员无法审计用户操作行为，难以排查历史问题或保障数据安全。若部署到实际环境，系统需要引入日志模块以记录访问日志、异常日志等关键信息。

6. 部分模块未完全对接前端：例如教师端成绩统计分析、学生课程推荐等功能在后端已有接口设计，但因时间限制未完成前端展示逻辑，整体功能体验不够完整。

7. 缺乏单元测试与接口自动化测试：项目中多数功能依赖人工测试，未引入自动化测试框架，缺乏接口测试覆盖率保证，后期维护成本较高。

7.3 进一步改进思路和体会

为进一步提升本系统的功能完整性、系统稳定性及用户体验，后续可从以下几个方面进行优化与扩展：

1. 引入统一身份认证与权限管理框架：当前系统采用简单的登录验证机制，后续可集成 OAuth2 或 JWT (JSON Web Token) 等标准认证方案，对不同用户角色（如管理员、教师、学生）进行细粒度权限划分，提升系统安全性。

2. 丰富前端交互体验：利用前端框架（如 Vue.js）进一步完善表格组件功能，如列排序、模糊搜索、条件筛选、分页缓存等，提高数据展示效率和用户操作流畅度。

3. 实现数据可视化模块：结合后端提供的统计接口，引入图表组件（如 ECharts），直观展示学生成绩分布、选课情况、生源地分布等信息，提升系统的信息传达效果。

4. 增强系统健壮性与可维护性：补充接口层和数据库层的异常处理机制，引入统一错误响应格式，同时通过单元测试与接口自动化测试保障系统稳定运行。

5. 优化数据库结构与查询性能：对高频访问的表字段建立合理索引，优化查询语句与分页逻辑，必要时考虑缓存机制（如 Redis）以减少数据库压力。

6. 拓展更多实际业务功能：如教师端成绩导入、导出 Excel 报表功能，学生端学分预警、课程推荐、教学评价反馈模块等，提升系统服务价值。

7. 集成后台操作日志与审计功能：记录用户关键操作与异常行为，便于后期审查、排查问题，增强系统的数据安全与可追溯性。

7.4 致谢

在本次系统的设计与开发过程中，我深切体会到团队协作与知识共享的重要性。衷心感谢所有给予我帮助和支持的老师、同学及合作伙伴们。指导老师的悉心教导和宝贵建议，不仅帮助我更好地将理论知识应用于实践，更为项目的顺利推进指明了方向。老师们严谨的治学态度和丰富的工程经验，让我受益匪浅，这些都将为我未来学习和工作中的宝贵财富。

团队成员们的紧密配合与高效协作是项目成功的关键。在系统设计与实现过程中，大家积极讨论、集思广益，共同攻克技术难题。每一次的思维碰撞都带来了新的启发，每一个问题的解决都凝聚着集体的智慧。特别感谢测试阶段提供宝贵意见的同学们，你们的细致测试和真诚反馈帮助我们发现并改进了系统中的诸多细节，使最终成果更加完善。

最后，要感谢学院提供的优质学习平台和丰富资源。完善的实验环境和专业的课程设置，为我们创造了良好的实践机会，让理论知识得以落地生根。这段宝贵的开发经历不仅提升了我的专业技能，更培养了我解决复杂问题的能力，为未来的职业发展奠定了坚实基础。

再次向所有给予关心、支持和帮助的人们致以最诚挚的谢意！